IE University

Bachelor in Computer Science and Artificial Intelligence

Improving and Automating "QR Forge" with DevOps Practices

Individual Assignment 2 – Software Design, DevOps and Operations

Student: Juan Sebastián Peña
Course: SDDO – 2025
Topic: Improve and Automate Your Application with DevOps
App: QR Forge (FastAPI QR Generator)
URL: jsebastianqrapp.azurewebsites.net

Date: November 30, 2025

1 Introduction

The goal of this assignment was to take the QR Forge application from the first project and turn it into a more professional, automated and observable system. The original app is a FastAPI service that lets users generate, save and download customised QR codes, with authentication, history and export features. In this second part the focus moved from "just making it work" to making it reliable: code quality, automated tests and coverage, CI/CD pipelines, containerisation, cloud deployment and monitoring.

The final system runs as a Docker container in Azure Web App for Containers, pulling images from Azure Container Registry. A CI pipeline runs linting, tests and coverage checks on each change, while a CD pipeline builds the container, runs Alembic migrations and deploys the image. The app exposes a health endpoint for probes and a Prometheus metrics endpoint for monitoring. Together these pieces form a small but realistic DevOps setup.

2 Codebase Improvements

2.1 Restructuring the Project

At the start, the project still felt like a student prototype. Several modules lived at the top level, some responsibilities blurred across files and certain paths assumed local development defaults. The first improvement was a structural one: the code was reorganised into a clear `app/` package.

The new structure groups the main concerns:

- **Entry points:** `app.main` builds the FastAPI application and wires routers, while `app.server` is the ASGI entry used by `uvicorn` in Docker.
- **Configuration:** `app.config` centralises environment variables such as `SECRET_KEY`, `POSTGRES_URL`, `DATABASE_URL`, `QR_ASSETS_DIR` and `QR_TEMP_DIR`.
- **Persistence:** `app.db` manages the SQLModel engine and sessions, with specific logic for PostgreSQL versus SQLite.
- **Domain logic:** models live in `app.models`, schemas in `app.schemas`, and business logic in service modules under `app.services`.
- **API surface:** routers such as `auth`, `user`, `qr` and `export` sit under `app.routers`.

This reorganisation removes a lot of "mystery wiring". Each folder now has a clear purpose, which makes navigation easier and helps tools like tests and linters give more precise feedback.

2.2 Configuration and Secret Handling

Another early clean-up focused on configuration. All sensitive or environment-specific values are now read from environment variables through a single settings object. The most important change is the way the database URL is resolved:

- If `POSTGRES_URL` is set, the app uses that value for production.
- Otherwise it falls back to `DATABASE_URL`, which defaults to a SQLite file under `~/data/qrcodes.db`.

This small change has a big impact. It allows a fast local workflow with SQLite while still targeting PostgreSQL in Azure just by setting one environment variable. Hardcoded values disappeared and any credentials now live in Azure App Settings and GitHub secrets instead of inside the repository.

All asset and temporary folders were also redirected to writable locations under `/home/app/data` in the container. This prevents permission errors when the non-root application user writes PNG and SVG files in production.

2.3 Database and Resource Management

The database layer needed to support two quite different runtime environments: tiny in-memory SQLite databases for testing and the managed PostgreSQL instance in Azure. The engine factory now handles these paths explicitly.

When the URL starts with `sqlite`, the app ensures that the target directory exists and enables the `check_same_thread=False` flag so that tests can run in parallel with FastAPI. When the URL points to PostgreSQL, the engine no longer attempts any filesystem operations and instead relies on connection pooling options and standard SQLAlchemy behaviour. This separation removes subtle bugs and makes the intent obvious.

In addition, the health check endpoint no longer depends on complex ORM interactions. It uses a light query for its optional database check and returns a simple JSON payload that works reliably for both local and cloud probes.

2.4 Logging, Error Handling and Clean-up

The project originally used informal `print` statements sprinkled in different places. These have been replaced by structured logging with a configurable `LOG_LEVEL`. Each log entry now carries at least a timestamp, a level, a module and a message. This change might look cosmetic, yet it becomes invaluable when debugging a failing deployment or tracking down a performance issue.

Several small clean-ups supported this shift:

- Exception paths log descriptive messages instead of failing silently.
- Dead or unused code paths were removed or simplified.
- Responsibility lines between routers and services were tightened so that HTTP endpoints delegate quickly to testable functions.

The overall effect is a codebase that behaves more predictably and exposes clearer signals when things go wrong.

3 Testing Strategy and Coverage

3.1 Unit and Integration Tests

The test suite covers both individual units and full flows through the API. Unit tests check services such as QR generation, authentication, password hashing and schema validation. These tests run against small, isolated components and use in-memory databases where convenient.

Integration tests use FastAPI's `TestClient` to exercise realistic scenarios:

- A user signs up, logs in and receives a JWT.
- The user creates one or more QR codes with different configuration options.
- The test verifies that entries appear in the database and that temporary files exist where expected.
- The user downloads a QR code and then deletes it, and the test confirms that records and files disappear.

By simulating what a real user does, these tests give much more confidence than only checking isolated functions.

3.2 Coverage Gate and Test Reports

The CI pipeline enforces a minimum coverage of 70%. Locally, coverage stays far above that threshold, around the ninety-percent range. The important detail is not the exact number but the fact that coverage acts as a guardrail. If new code arrives without tests, the pipeline turns red instead of silently accepting the regression.

The coverage tool produces both human-readable reports and machine-readable files such as `coverage.xml`. These can be used later for dashboards, quality gates or trend tracking. A separate test report collects results from `pytest` so that failures are easier to inspect over time.

4 Continuous Integration Pipeline

4.1 Pipeline Overview

The continuous integration pipeline lives in `.github/workflows/ci.yaml`. It triggers on relevant events such as pushes and pull requests. Each run follows a clear sequence:

1. Check out the repository.
2. Set up the Python environment with the required version.
3. Install runtime dependencies from `requirements.txt` and development tools from `requirements-dev.txt`.

4. Run linting with `ruff` and formatting checks with `black`.

5. Execute the test suite with `pytest` under coverage.

6. Enforce a coverage threshold with `coverage report --fail-under=70`.

7. Build a Docker image as a final verification step.

8. Run a Trivy scan on the image to catch common vulnerabilities.

Each stage protects the next one. If linting fails, tests do not run. If coverage drops below the threshold, the build step never starts. That simple ordering prevents the classic situation where a broken main branch goes unnoticed until someone tries to deploy.

4.2 Quality and Security Checks

The pipeline takes security and quality seriously for a student project:

- Linting prevents typical mistakes, from unused imports to risky patterns.
- Formatting checks keep a consistent style, which makes diffs and reviews easier.
- The Trivy scan looks for known vulnerabilities in the base image and dependencies.

While this stack is relatively lightweight, it already mirrors what many teams do in production. It shows how automated checks can remain fast yet still provide real value.

5 Deployment Automation and Azure Setup

5.1 Azure Resources

The application lives in a small Azure environment composed of four main resources:

Resource	**Name**	**Role**
Resource Group	BCSAI2025-DEVOPS-STUDENTS-A	Logical container for all project resources.
Container Registry	jsebastianacr	Stores Docker images for the application.
Web App for Containers	jsebastianqrapp	Hosts the containerised FastAPI app.
PostgreSQL Database	jsebastianqrdb	Provides the managed database used in production.

The Web App pulls images from the registry through a user-assigned managed identity with the `AcrPull` role. This avoids storing container registry credentials directly in the app configuration.

Application settings in the Web App define secrets and environment values such as `SECRET_KEY`, `ACCESS_TOKEN_EXPIRE_MINUTES`, `POSTGRES_URL`, `DATABASE_URL`, `QR_ASSETS_DIR` and `QR_TEMP_DIR`. Many of these map directly to the configuration object in `app.config`.

5.2 Continuous Deployment Pipeline

The CD pipeline lives in `.github/workflows/cd.yaml` and runs on pushes to the main branch or manual triggers. Its steps are more involved than CI because they interact with Azure:

1. **Authenticate to Azure.** The pipeline logs in using a GitHub Actions service principal.
2. **Build and push the image.** It builds the Docker image for the current commit, tags it with both the SHA and `:latest`, and pushes it to `jsebastianacr`.
3. **Run migrations from the image.** Before deploying, the pipeline runs `alembic upgrade head` inside the freshly built image, using the `POSTGRES_URL` secret. This ensures the schema matches the application code.
4. **Deploy to the Web App.** The pipeline updates the Web App to point at the new image tag in the registry.
5. **Smoke-test the deployment.** A small script queries Azure to discover the application's hostname and then calls the `/health` endpoint several times with retries. Only if the response is successful does the job finish as passed.

Running migrations from the same image that will later serve traffic is a deliberate choice. It reduces the risk of version drift between the code that writes migrations and the code that the server executes.

5.3 Secrets and Branch Protection

Only the main branch has automatic deployment. Other branches can still trigger the CD workflow manually for testing, but there is a clear boundary between day-to-day experimentation and production updates.

Secrets such as the PostgreSQL connection string live in GitHub repository secrets and in Azure App Settings, not inside the pipeline YAML. That separation means that if access to the repository is later reused or shared, no raw credentials appear in the configuration files.

6 Monitoring, Health Checks and Observability

6.1 Health Endpoint

The `/health` endpoint returns a compact JSON payload with a status field. The Azure Web App uses this endpoint for its health probes, and the CD pipeline uses it for smoke testing before considering a deployment successful.

This design keeps health checks simple enough that they are very unlikely to break. The logic focuses on a few "can the app respond at all?" checks instead of performing heavy computations. When needed, the endpoint can also include a lightweight database connectivity check, which helps detect configuration problems early.

6.2 Prometheus Metrics and Local Monitoring Stack

The application exposes a `/metrics` endpoint compatible with Prometheus. A metrics instrumentator records:

- Request counts per endpoint and HTTP method.
- Latency distributions through histograms.
- Error rates by status code.

In addition to the endpoint, the repository contains a small monitoring stack under `monitoring/`. A Docker Compose file spins up Prometheus and Grafana alongside the app. A prebuilt Grafana dashboard reads from the Prometheus scrape target and visualises common indicators: latency, throughput and error spikes.

Although this stack runs locally rather than in Azure, it demonstrates how to wire metrics, scraping and dashboards together. The same pattern could later be replicated using managed services.

6.3 Logging and Diagnostics

Structured logs complement metrics. When something goes wrong during deployment, the combination of Azure Web App logs, CI/CD logs and application logs quickly narrows down the cause. For example, a wrong connection string usually produces a clear error message in the logs, which can then be traced back to a misconfigured environment variable.

7 Conclusions and Future Work

This assignment turned the QR Forge application from a single-project repository into a small but credible cloud service. The project now uses a clean package structure, configuration through environment variables, a test suite with a coverage gate, CI and CD pipelines and basic monitoring and logging.

Several natural next steps remain for future iterations:

- Use Azure Key Vault or a similar secret manager instead of storing connection strings directly in App Settings.
- Add deployment slots for blue–green or canary releases with automatic rollback on failure.

- Extend monitoring with alert rules based on metrics and health checks.
- Add more negative tests for database failures, rate limits and corrupted assets.

Even without those extra features, the current setup already matches many real-world DevOps practices and provides a solid foundation for future experiments.

8 Use of AI Coding Assistants

8.1 Overview

Throughout this assignment several AI coding assistants supported the work: ChatGPT, GitHub Copilot (including its "workspace" or agent mode) and Codex-style suggestions inside the editor. Each tool played a slightly different role. Some helped shape the high-level plan; others generated concrete code, YAML and configuration snippets that were then reviewed, tested and adapted.

This section explains how those tools were used, why they were useful and which precautions were taken to use them responsibly.

8.2 Using ChatGPT for Design, Refactoring and DevOps Decisions

ChatGPT acted as a kind of remote teammate focused on architecture and DevOps questions. The interaction typically followed a pattern:

- First, the existing repository layout, goals and grading rubric were described in plain language.
- Then, specific pain points were discussed: how to handle PostgreSQL versus SQLite, how to structure the CI/CD pipelines or how to expose metrics.
- ChatGPT proposed refactor plans, step-by-step CI/CD flows and ideas for monitoring and documentation.

For example, it suggested moving to a clear `app/` package, making configuration purely environment-driven and building a CI pipeline that enforces a coverage gate and builds a test image. It also helped design the CD pipeline so that Alembic migrations run inside the same container image that later serves traffic, and so that a smoke test hits the real `/health` endpoint discovered via Azure CLI.

Another useful area was documentation. ChatGPT helped draft sections of the README and this report, especially in describing the pipeline stages and summarising trade-offs. All drafts were reviewed and edited by hand to fit the assignment's tone and avoid generic phrasing.

8.3 Using GitHub Copilot in Workspace / Agent Mode

GitHub Copilot's workspace or "agent" mode was used as a powerful automation layer on top of the repository. After feeding a detailed prompt that explained the goals—for example, "clean up the repo, separate runtime and dev dependencies, strengthen CI/CD, improve the Dockerfile and keep tests green"—the agent scanned the entire project and proposed concrete changes.

Typical changes included:

- Splitting dependencies into `requirements.txt` and `requirements-dev.txt`.
- Updating `ci.yaml` to add caching, coverage gates and Trivy scanning.

- Improving the Dockerfile to use a multi-stage build, run as a non-root user and add a health check.
- Adjusting the CD workflow to build and push images, run migrations in the container and perform smoke tests before finishing.

Each proposed change went through a human review step. The diffs were read, the logic was checked and the tests were run locally. When something looked too complex or unnecessary for the scope of the assignment, it was simplified or reverted. In that sense, the agent acted more like a very fast junior developer than an unquestioned authority.

8.4 Using Codex and Inline Suggestions

Codex-style inline suggestions inside the editor helped with smaller details: writing short functions, fixing imports after moving modules around, adding logging statements or filling out repetitive parts of YAML files. These suggestions sped up routine tasks but still required careful reading.

For instance, when the project moved to the `app/` package, many import paths broke. Inline suggestions quickly proposed the correct `from app.routers import ...` statements. The same happened when defining environment variable defaults or configuring the FastAPI application factory.

The key pattern was always the same: accept suggestions that clearly matched the project's direction and reject anything that seemed unfamiliar, overcomplicated or inconsistent with previous decisions.

8.5 Good Practices and Risk Mitigation

Using AI assistants in a DevOps-heavy assignment raises legitimate questions about confidentiality and reliability. Several rules were followed to use these tools responsibly:

- **No confidential secrets in prompts.** Real passwords, full connection strings with real credentials and other sensitive values were never pasted into prompts. When discussing database URLs or API keys, dummy placeholders were used instead.
- **Environment variables instead of hardcoding.** The code was designed so that secrets come from environment variables. That made it easier to talk about configuration patterns without exposing real values.
- **Human review of all generated code.** No snippet, Dockerfile or workflow was accepted blindly. Every change was read, understood and adapted if needed. Tests ran after relevant changes to catch regressions.
- **Scope control.** AI tools were asked to focus on the existing repository. They were not given access to unrelated projects or private information beyond what was necessary for this assignment.

- **Transparency.** The use of AI assistance is documented in this section so that the learning process and the contribution of these tools remain clear.
- **Avoiding over-dependence.** Design decisions—such as how to structure the app, which Azure resources to use or which trade-offs to accept—were made consciously instead of outsourcing them entirely to AI prompts.

These practices help balance the advantages of AI (speed, ideas, boilerplate generation) with the responsibility to protect data and maintain control over the codebase.

8.6 Reflections on Learning

Working with AI assistants in this assignment felt similar to pairing with an experienced but sometimes over-enthusiastic teammate. They are excellent at spotting patterns, suggesting refactors and generating full pipeline files in seconds. At the same time, they occasionally propose overly complex solutions or introduce assumptions that do not match the actual environment.

The most valuable part of the experience was learning how to use these tools critically. They proved especially helpful for:

- Clarifying DevOps concepts and turning them into concrete steps.
- Speeding up repetitive or mechanical code transformations.
- Generating first drafts of documentation and reports that could then be edited.

In the end, the assignment did not become "an AI project" but a DevOps project where AI played a supportive role. The final system reflects a combination of human understanding, course content and AI-accelerated implementation.

- Project Codebase
  - Root
    - .pre-commit-config.yaml
    - export_codebase.py
    - qr.db
  - .github
  - .github\workflows
    - cd.yaml
    - ci.yaml
  - .vscode
    - settings.json
  - alembic
    - env.py
  - alembic\versions
    - 6335c06e104d_initial_schema.py
  - app
    - init.py
    - config.py
    - db.py
    - main.py
    - models.py
    - schemas.py
    - server.py
    - storage.py
  - app\assets
  - app\assets\icons
  - app\core
    - init.py
    - bootstrap.py
    - logging.py
    - security.py
  - app\generated_pngs
  - app\generated_svgs
  - app\home
    - init.py
  - app\home\site
  - app\home\site\wwwroot

- qrcodes.db
- app\monitoring
  - init.py
  - docker-compose.monitoring.yml
  - grafana-dashboard.json
  - prometheus.yml
- app\routers
  - init.py
  - auth.py
  - export.py
  - qr.py
  - user.py
- app\services
  - init.py
  - auth.py
  - qr.py
  - qr_items.py
  - user.py
- app\static
- app\templates
  - base.html
  - history.html
  - home.html
  - index.html
  - login.html
  - profile.html
  - signup.html
- Assignment1
- Assignment1\report
- Assignment1\report\annex
- Assignment1\report\annex\wireframe-mockup
- Assignment2
- logs
- tests
  - conftest.py
  - test_alembic_import.py
  - test_auth.py
  - test_auth_and_qr.py

- test_auth_negative_extra.py
- test_db_failure.py
- test_health_db_failure.py
- test_integration_flow.py
- test_qr.py
- test_qr_service_unit.py
- test_schemas.py
- test_security.py
- test_services_exceptions.py

Project Codebase

📂 Root

.pre-commit-config.yaml

```
repos:
  - repo: https://github.com/charliermarsh/ruff-pre-commit
    rev: v0.18.0
    hooks:
      - id: ruff
  - repo: https://github.com/psf/black
    rev: 23.9.1
    hooks:
      - id: black
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v4.4.0
    hooks:
      - id: end-of-file-fixer
      - id: trailing-whitespace
```

export_codebase.py

```
import os
from pathlib import Path

# Project root (relative to this script)
PROJECT_ROOT = Path(".")
```

```
# Output file
OUTPUT_FILE = Path("codebase.md")

# Extensions to include
INCLUDE_EXTS = (".py", ".html", ".yml", ".yaml", ".json", ".db")

# Folders to skip
SKIP_DIRS = {".git", "__pycache__", ".venv", "node_modules", ".ruff_cache",
".pytest_cache"}


def should_skip(dirpath: Path) -> bool:
    parts = set(dirpath.parts)
    return bool(parts & SKIP_DIRS)


def main() -> None:
    with OUTPUT_FILE.open("w", encoding="utf-8") as out:
        out.write("# Project Codebase\n")
        for root, dirs, files in os.walk(PROJECT_ROOT):
            root_path = Path(root)
            if should_skip(root_path):
                continue

            rel_path = root_path.relative_to(PROJECT_ROOT)
            rel_label = "Root" if str(rel_path) == "." else str(rel_path)
            out.write(f"\n\n## 📁 {rel_label}\n\n")

            for file in sorted(files):
                if not file.endswith(INCLUDE_EXTS):
                    continue
                filepath = root_path / file
                ext = filepath.suffix.lstrip(".") or ""
                out.write(f"\n### 📄 {file}\n\n")
                out.write(f"```{ext}\n")
                try:
                    with filepath.open("r", encoding="utf-8") as f:
                        out.write(f.read())
                except Exception as e:
                    out.write(f"⚠️ Could not read file: {e}")
                out.write("\n```\n")

    print(f"✅ Codebase exported to {OUTPUT_FILE}. Now run (optional):")
    print("   pandoc codebase.md -o codebase.pdf   # convert to PDF")


if __name__ == "__main__":
    main()
```

qr.db

⚠️ Could not read file: 'utf-8' codec can't decode byte 0x86 in position 98: invalid start byte

📁 .github

📁 .github\workflows

📄 cd.yaml

```yaml
name: CD

on:
  push:
    branches:
      - main
  workflow_dispatch: {}

env:
  IMAGE_TAG: ${{ github.sha }}

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Validate required secrets
        run: |
          if [ -z "${{ secrets.AZURE_CREDENTIALS }}" ] || \
             [ -z "${{ secrets.ACR_NAME }}" ] || \
             [ -z "${{ secrets.AZURE_RESOURCE_GROUP }}" ] || \
             [ -z "${{ secrets.AZURE_WEBAPP_NAME }}" ] || \
             [ -z "${{ secrets.POSTGRES_URL }}" ]; then
            echo "Missing required secrets: AZURE_CREDENTIALS, ACR_NAME,
AZURE_RESOURCE_GROUP, AZURE_WEBAPP_NAME, POSTGRES_URL" >&2
            exit 1
          fi

      - name: Azure Login
        uses: azure/login@v2
        with:
          creds: ${{ secrets.AZURE_CREDENTIALS }}

      - name: ACR Login
        run: az acr login --name ${{ secrets.ACR_NAME }}
```

```
      - name: Build & push image to ACR (az acr build)
        run: |
          az acr build --registry ${{ secrets.ACR_NAME }} --image "${{
secrets.AZURE_WEBAPP_NAME }}:${{ env.IMAGE_TAG }}" .

      - name: Ensure staging slot exists (idempotent)
        run: echo "Skipping slot creation; deploying direct to production"

      - name: Configure production container image
        run: |
          az webapp config appsettings set \
            --resource-group ${{ secrets.AZURE_RESOURCE_GROUP }} \
            --name ${{ secrets.AZURE_WEBAPP_NAME }} \
            --settings POSTGRES_URL="${{ secrets.POSTGRES_URL }}"
          IMAGE="${{ secrets.ACR_NAME }}.azurecr.io/${{ secrets.AZURE_WEBAPP_NAME
}}:${{ env.IMAGE_TAG }}"
          az webapp config container set \
            --resource-group ${{ secrets.AZURE_RESOURCE_GROUP }} \
            --name ${{ secrets.AZURE_WEBAPP_NAME }} \
            --container-image-name "$IMAGE"

      - name: Run alembic migrations from built image
        run: |
          IMAGE="${{ secrets.ACR_NAME }}.azurecr.io/${{ secrets.AZURE_WEBAPP_NAME
}}:${{ env.IMAGE_TAG }}"
          az acr login --name ${{ secrets.ACR_NAME }}
          docker pull "$IMAGE"
          docker run --rm --entrypoint "" -e POSTGRES_URL="${{ secrets.POSTGRES_URL
}}" "$IMAGE" sh -c "alembic upgrade head"

      - name: Smoke test production
        run: |
          HOST=$(az webapp show --resource-group ${{ secrets.AZURE_RESOURCE_GROUP
}} --name ${{ secrets.AZURE_WEBAPP_NAME }} --query defaultHostName -o tsv)
          if [ -z "$HOST" ] || [ "$HOST" = "null" ]; then
            echo "ERROR: could not discover production host" >&2
            exit 1
          fi
          APP_URL="https://$HOST/health"
          echo "Checking $APP_URL"
          for i in $(seq 1 15); do
            status=$(curl -s -o /dev/null -w "%{http_code}" --max-time 10
"$APP_URL" || echo "000")
            echo "Attempt $i: $status"
            if [ "$status" = "200" ]; then
              echo "Production healthy"
              break
            fi
            sleep 5
          done
          if [ "$status" != "200" ]; then
            echo "Production smoke test failed after 15 attempts" >&2
            exit 1
          fi

      - name: Post-deploy verification
```

```
        run: |
          HOST=$(az webapp show --resource-group ${{ secrets.AZURE_RESOURCE_GROUP
}} --name ${{ secrets.AZURE_WEBAPP_NAME }} --query defaultHostName -o tsv)
          if [ -z "$HOST" ] || [ "$HOST" = "null" ]; then
            echo "ERROR: could not discover production host" >&2
            exit 1
          fi
          PROD_URL="https://$HOST/health"
          echo "Checking $PROD_URL"
          curl -f "$PROD_URL"
```

ci.yaml

```yaml
name: CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

env:
  COVERAGE_GATE: 70

jobs:
  test-and-lint:
    runs-on: ubuntu-latest
    env:
      DATABASE_URL: sqlite://
    strategy:
      matrix:
        python-version: [3.11]

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: ${{ matrix.python-version }}

      - name: Cache pip
        uses: actions/cache@v4
        with:
          path: ~/.cache/pip
          key: ${{ runner.os }}-pip-${{ hashFiles('**/requirements*.txt') }}
          restore-keys: |
            ${{ runner.os }}-pip-

      - name: Install deps
```

```
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
          pip install -r requirements-dev.txt

      - name: Lint (ruff)
        run: ruff check .

      - name: Format check (black)
        run: black --check .

      - name: Run tests with coverage
        run: |
          coverage run -m pytest --junitxml=results.xml
          coverage xml -o coverage.xml

      - name: Enforce coverage threshold
        run: |
          coverage report --fail-under=${{ env.COVERAGE_GATE }}

  container-scan-smoke:
    name: Container build, Trivy scan and smoke tests
    runs-on: ubuntu-latest
    needs: test-and-lint
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v2

      - name: Build Docker image
        run: docker build -t qr_forge:ci .

      - name: Trivy scan (image)
        uses: aquasecurity/trivy-action@0.21.0
        with:
          image-ref: qr_forge:ci
          format: 'json'
          output: trivy-report.json
          severity: CRITICAL,HIGH

      - name: Upload Trivy report
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: trivy-report
          path: trivy-report.json

      - name: Run container for smoke tests
        run: |
          docker run -d --name qrforge_ci -p 8000:8000 -e
DATABASE_URL="sqlite:///tmp/qrcodes.db" qr_forge:ci
          retries=15
          while [ $retries -gt 0 ]; do
            if curl --silent --fail http://localhost:8000/health; then
              echo "health endpoint OK"
```

```
            break
          fi
          retries=$((retries-1))
          sleep 2
        done
        if [ $retries -le 0 ]; then
          echo "health endpoint did not become ready";
          docker logs qrforge_ci || true;
          exit 1
        fi
        curl --silent --fail http://localhost:8000/metrics | head -n 40

    - name: Stop and remove container
      if: always()
      run: |
        docker rm -f qrforge_ci || true

    # Container build, Trivy scan and smoke tests moved to dedicated job
```

📁 .vscode

📄 settings.json

```
{
    "python.testing.pytestArgs": [
        "tests"
    ],
    "python.testing.unittestEnabled": false,
    "python.testing.pytestEnabled": true
}
```

📁 alembic

📄 env.py

```
import os
from logging.config import fileConfig

from sqlalchemy import create_engine
from sqlmodel import SQLModel
```

```python
# Import project models only to populate metadata; avoid importing application
# config or db modules which may have side effects (like creating directories).
import app.models  # noqa: F401
from alembic import context

# This is the Alembic Config object. It provides access to the
# values within the .ini file in use.
config = context.config

# Interpret the config file for Python logging.
if config.config_file_name is not None:
    fileConfig(config.config_file_name)

# pick up the metadata from your models (models import ensures classes are
registered)
target_metadata = SQLModel.metadata


def get_url() -> str:
    """Return the DB URL to use for migrations.

    Preference order:
    1. `POSTGRES_URL` environment variable (used by CI/CD)
    2. `sqlalchemy.url` value from `alembic.ini`
    3. `DATABASE_URL` environment variable

    If none are set, raise an explicit error to avoid silent defaults that may
    attempt to create directories under restricted paths.
    """
    url = os.getenv("POSTGRES_URL")
    if url:
        return url
    url = config.get_main_option("sqlalchemy.url")
    if url:
        return url
    url = os.getenv("DATABASE_URL")
    if url:
        return url
    raise RuntimeError(
        "No database URL configured for Alembic. Set POSTGRES_URL or"
        " sqlalchemy.url in alembic.ini"
    )


def run_migrations_offline() -> None:
    url = get_url()
    context.configure(
        url=url,
        target_metadata=target_metadata,
        literal_binds=True,
        dialect_opts={"paramstyle": "named"},
    )

    with context.begin_transaction():
        context.run_migrations()
```

```python
def run_migrations_online() -> None:
    url = get_url()
    connect_args = {}
    if url.startswith("sqlite"):
        connect_args = {"check_same_thread": False}
    connectable = create_engine(url, echo=False, connect_args=connect_args)

    with connectable.connect() as connection:
        context.configure(connection=connection, target_metadata=target_metadata)

        with context.begin_transaction():
            context.run_migrations()


if context.is_offline_mode():
    run_migrations_offline()
else:
    run_migrations_online()
```

📂 alembic\versions

📄 6335c06e104d_initial_schema.py

```python
"""initial schema

Revision ID: 6335c06e104d
Revises:
Create Date: 2025-11-29 23:25:47.767549

"""

from typing import Sequence, Union

# revision identifiers, used by Alembic.
revision: str = "6335c06e104d"
down_revision: Union[str, None] = None
branch_labels: Union[str, Sequence[str], None] = None
depends_on: Union[str, Sequence[str], None] = None


def upgrade() -> None:
    # ### commands auto generated by Alembic - please adjust! ###
    pass
    # ### end Alembic commands ###


def downgrade() -> None:
    # ### commands auto generated by Alembic - please adjust! ###
```

```
    pass
    # ### end Alembic commands ###
```

app

init.py

```
"""Application package initializer."""

from app.main import app

__all__ = ["app"]
```

config.py

```
"""Application configuration.

Centralises environment-driven settings. Supports Key Vault-style references
in environment variables (the App Service Key Vault reference syntax is
parsed at runtime by `app.py` which will attempt to resolve secrets from
Azure Key Vault when available).
"""

import logging
import os
from dataclasses import dataclass, field
from functools import lru_cache
from pathlib import Path
from typing import Optional

from dotenv import load_dotenv

load_dotenv()

logger = logging.getLogger("qr_forge.config")
BASE_DIR = Path(__file__).resolve().parent
# Writable defaults under the user's home (works locally and in Azure)
HOME_DIR = Path.home()


@dataclass
class Settings:
    """Runtime configuration pulled from environment variables.
```

```
    This dataclass centralises environment-driven configuration. It prefers
    `POSTGRES_URL` (for production) and falls back to `DATABASE_URL` which
    may point to a local sqlite file for development and tests.
    """

    secret_key: str = field(
        default_factory=lambda: os.getenv("SECRET_KEY", "change-me-in-env")
    )
    access_token_expire_minutes: int = field(
        default_factory=lambda: int(os.getenv("ACCESS_TOKEN_EXPIRE_MINUTES",
"720"))
    )
    algorithm: str = field(default_factory=lambda: os.getenv("ALGORITHM", "HS256"))
    postgres_url: Optional[str] = field(
        default_factory=lambda: os.getenv("POSTGRES_URL")
    )
    database_url: str = field(
        default_factory=lambda: os.getenv(
            "DATABASE_URL", f"sqlite:///{(HOME_DIR / 'data' /
'qrcodes.db').as_posix()}"
        )
    )
    assets_dir: Path = field(
        default_factory=lambda: Path(
            os.getenv("QR_ASSETS_DIR", str(HOME_DIR / "data" / "qr_assets"))
        )
    )
    temp_dir: Path = field(
        default_factory=lambda: Path(
            os.getenv("QR_TEMP_DIR", str(HOME_DIR / "data" / "qr_temp"))
        )
    )
    # Optional telemetry and observability configuration
    app_insights_connection_string: Optional[str] = field(
        default_factory=lambda: os.getenv("APPINSIGHTS_CONN")
    )

    def __post_init__(self) -> None:
        # Use POSTGRES_URL when available, otherwise DATABASE_URL
        self.database_url = self.postgres_url or self.database_url
        if not self.database_url:
            logger.warning(
                "No database URL configured; application may not function
correctly"
            )
        # Normalise paths for downstream usage; directory creation should be
        # performed at application startup by calling `ensure_dirs(settings)`
        self.assets_dir = Path(self.assets_dir)
        self.temp_dir = Path(self.temp_dir)


@lru_cache(maxsize=1)
def get_settings() -> Settings:
    """Return cached Settings instance."""
    return Settings()
```

```
settings = get_settings()
```

db.py

```
"""Database helpers and engine configuration.

This module centralises engine creation and session management. It configures
connection pooling for production databases and keeps an SQLite-friendly
fallback for local development and tests.
"""

from collections.abc import Generator
import logging
from pathlib import Path
from typing import Optional

from sqlalchemy.engine import make_url
from sqlmodel import Session, SQLModel, create_engine

from app.config import settings

logger = logging.getLogger("qr_forge.db")


def _ensure_sqlite_dir(database_url: str) -> None:
    """Ensure the parent directory for a sqlite file exists."""
    if not database_url.startswith("sqlite"):
        return
    url = make_url(database_url)
    if url.database:
        Path(url.database).parent.mkdir(parents=True, exist_ok=True)


def get_engine(database_url: Optional[str] = None):
    """Create and return a SQLAlchemy engine configured for the provided URL.

    Production Postgres engines enable pooling and pre-ping to avoid stale
    connections. For sqlite we set `check_same_thread` so the file-based DB
    works in multi-threaded test scenarios.
    """
    url = database_url or settings.database_url
    if not url:
        raise RuntimeError("No database URL provided; set POSTGRES_URL or
DATABASE_URL")
    is_sqlite = url.startswith("sqlite")
    if is_sqlite:
        _ensure_sqlite_dir(url)
    # Default connect_args for sqlite
    connect_args = {"check_same_thread": False} if is_sqlite else {}
```

```
    # Pooling options for production DBs
    engine_kwargs = {
        "echo": False,
    }
    if not is_sqlite:
        # Production Postgres pooling and resiliency
        engine_kwargs.update(
            {
                "pool_pre_ping": True,
                "pool_size": 5,
                "max_overflow": 10,
            }
        )
        # If the connection string does not specify sslmode and we're on Azure
        # enforce sslmode=require by appending the parameter.
        if "sslmode" not in url and url.startswith("postgres"):
            if "?" in url:
                url = f"{url}&sslmode=require"
            else:
                url = f"{url}?sslmode=require"

    logger.debug("Creating engine for URL: %s (sqlite=%s)", url, is_sqlite)
    return create_engine(url, connect_args=connect_args, **engine_kwargs)


engine = get_engine()


def init_db(db_engine=None) -> None:
    """Initialize database schema (create tables) on the provided engine."""
    active_engine = db_engine or engine
    SQLModel.metadata.create_all(active_engine)


def get_session() -> Generator[Session, None, None]:
    """Yield a SQLModel session for use in dependencies and services."""
    with Session(engine) as session:
        yield session
```

main.py

```
from __future__ import annotations

import logging
from pathlib import Path
from contextlib import asynccontextmanager

from fastapi import FastAPI, Request, status
from fastapi.responses import FileResponse, HTMLResponse, PlainTextResponse
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
```

```
from app.config import settings
from app.core.bootstrap import ensure_dirs
from app.core.logging import configure_logging, get_logger
from app.db import init_db
from app.routers import auth, export, qr, user

# Prometheus metrics (use prometheus_client directly to ensure /metrics works in
prod)
try:
    from prometheus_client import (
        CONTENT_TYPE_LATEST,
        Counter,
        Histogram,
        generate_latest,
    )
except Exception:
    # Fallback no-op implementations so tests and environments without
    # prometheus_client can still import the application module.
    class _NoopMetric:
        def __init__(self, *args, **kwargs):
            pass

        def labels(self, *args, **kwargs):
            return self

        def inc(self, *args, **kwargs):
            return None

        def observe(self, *args, **kwargs):
            return None

    Counter = _NoopMetric
    Histogram = _NoopMetric

    def generate_latest():
        return b""

    CONTENT_TYPE_LATEST = "text/plain; version=0.0.4; charset=utf-8"

BASE_DIR = Path(__file__).parent

TAGS_METADATA = [
    {
        "name": "auth",
        "description": "Authentication endpoints: signup, login and logout",
    },
    {
        "name": "users",
        "description": "Profile endpoints for viewing, updating, or deleting a
user",
    },
    {
        "name": "qr",
        "description": "QR generation, preview, download and history for a user",
    },
    {
```

```
        "name": "export",
        "description": "CSV export of a user's QR history",
    },
]

app = FastAPI(
    title="QR Forge",
    description=(
        "Generate, preview, customise and manage QR codes locally with FastAPI."
    ),
    version="1.0.0",
    contact={
        "name": "QR Forge",
        "url": "https://github.com/",
        "email": "support@example.com",
    },
    license_info={"name": "MIT", "url": "https://opensource.org/licenses/MIT"},
    openapi_tags=TAGS_METADATA,
    docs_url="/docs",
    redoc_url=None,
)

# Prometheus metrics definitions
REQUEST_COUNT = Counter(
    "qr_forge_requests_total",
    "Total HTTP requests",
    ["method", "endpoint", "http_status"],
)
REQUEST_LATENCY = Histogram(
    "qr_forge_request_latency_seconds",
    "Request latency seconds",
    ["method", "endpoint"],
)


@app.middleware("http")
async def metrics_middleware(request: Request, call_next):
    path = request.url.path
    method = request.method
    if path == "/metrics":
        return await call_next(request)
    import time

    start = time.time()
    try:
        response = await call_next(request)
        status = str(response.status_code)
    except Exception:  # capture exceptions as 500
        status = "500"
        REQUEST_COUNT.labels(method=method, endpoint=path,
http_status=status).inc()
        raise
    finally:
        elapsed = time.time() - start
        try:
            REQUEST_LATENCY.labels(method=method, endpoint=path).observe(elapsed)
            REQUEST_COUNT.labels(method=method, endpoint=path,
```

```
http_status=status).inc()
        except Exception:
            pass
    return response


@app.get("/metrics", include_in_schema=False)
def metrics():
    """Prometheus metrics endpoint (always available)."""
    data = generate_latest()
    return PlainTextResponse(content=data, media_type=CONTENT_TYPE_LATEST)


def _setup_application_insights(app: FastAPI, logger: "logging.Logger") -> None:
    """Attempt to enable Application Insights/OpenTelemetry instrumentation.

    Imported lazily because these dependencies are optional in tests and
    some developer environments.
    """
    try:
        if not settings.app_insights_connection_string:
            return

        from opentelemetry import trace
        from opentelemetry.sdk.resources import Resource
        from opentelemetry.sdk.trace import TracerProvider
        from opentelemetry.sdk.trace.export import BatchSpanProcessor
        from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor

        resource = Resource.create({"service.name": "qr-forge"})
        provider = TracerProvider(resource=resource)

        # Try to create an exporter only if available; exporter packages are
        # optional in some environments so we handle ImportError gracefully.
        try:
            from azure.monitor.opentelemetry.exporter import (
                AzureMonitorTraceExporter,
            )

            exporter = AzureMonitorTraceExporter(
                connection_string=settings.app_insights_connection_string
            )
            provider.add_span_processor(BatchSpanProcessor(exporter))
        except Exception:
            # No exporter available; continue without span export
            logger.debug("No OTLP/Azure exporter available; skipping exporter
setup")

        trace.set_tracer_provider(provider)
        FastAPIInstrumentor().instrument_app(app)
        logger.info("Application Insights instrumentation enabled")
    except Exception:
        logger.exception("Failed to enable Application Insights instrumentation")


def _setup_prometheus_instrumentator(app: FastAPI, logger: "logging.Logger") ->
None:
```

```
    try:
        from prometheus_fastapi_instrumentator import Instrumentator

        Instrumentator().instrument(app).expose(app)
        logger.info("Prometheus instrumentation enabled (exposes /metrics)")
    except Exception:
        logger.info(
            "prometheus_fastapi_instrumentator not available; using builtin
metrics"
        )


@asynccontextmanager
async def app_lifespan(app: FastAPI):
    """Application lifespan context: configure logging, ensure dirs and DB.

    Replaces the older `on_event('startup')` approach and provides a single
    async context for startup and shutdown operations.
    """
    # Configure structured logging once at startup
    configure_logging()
    logger = get_logger("qr_forge.app")
    logger.info("Application startup: initializing DB and directories")

    # Ensure asset directories exist (moved out of import-time side-effects)
    created = ensure_dirs(settings)
    for p in created:
        logger.info("Ensured directory exists: %s", str(p))

    try:
        init_db()
    except Exception:
        logger.exception("init_db() raised an exception during startup")

    # Telemetry and metrics
    _setup_application_insights(app, logger)
    _setup_prometheus_instrumentator(app, logger)

    try:
        yield
    finally:
        logger.info("Application shutdown completed")


app.router.lifespan_context = app_lifespan


app.include_router(auth.router)
app.include_router(user.router)
app.include_router(qr.router)
app.include_router(export.router)

app.mount("/assets", StaticFiles(directory=str(BASE_DIR / "assets")),
name="assets")
app.mount("/static", StaticFiles(directory=str(BASE_DIR / "static")),
name="static")
```

```python
__all__ = ("app",)

templates = Jinja2Templates(directory=str(BASE_DIR / "templates"))


@app.get("/favicon.ico", include_in_schema=False)
def favicon() -> FileResponse:
    return FileResponse(BASE_DIR / "static" / "favicon.ico")


@app.get(
    "/", response_class=HTMLResponse, name="home", summary="Serve the landing page"
)
def home(request: Request) -> HTMLResponse:
    return templates.TemplateResponse(
        "home.html",
        {"request": request, "active_page": "home"},
    )


@app.get(
    "/generator",
    response_class=HTMLResponse,
    name="generator_page",
    summary="Serve the QR generator UI",
)
def generator_page(request: Request) -> HTMLResponse:
    return templates.TemplateResponse(
        "index.html",
        {"request": request, "active_page": "generator"},
    )


@app.get(
    "/history",
    response_class=HTMLResponse,
    name="history_page",
    summary="Serve the saved history UI",
)
def history_page(request: Request) -> HTMLResponse:
    return templates.TemplateResponse(
        "history.html",
        {"request": request, "active_page": "history"},
    )


@app.get(
    "/profile",
    response_class=HTMLResponse,
    name="profile_page",
    summary="Serve the profile management UI",
)
def profile_page(request: Request) -> HTMLResponse:
    return templates.TemplateResponse(
        "profile.html",
        {"request": request, "active_page": "profile"},
```

```
    )


@app.get(
    "/login",
    response_class=HTMLResponse,
    name="login_page",
    summary="Serve the login UI",
)
def login_page(request: Request) -> HTMLResponse:
    return templates.TemplateResponse(
        "login.html",
        {"request": request, "active_page": "login"},
    )


@app.get(
    "/signup",
    response_class=HTMLResponse,
    name="signup_page",
    summary="Serve the signup UI",
)
def signup_page(request: Request) -> HTMLResponse:
    return templates.TemplateResponse(
        "signup.html",
        {"request": request, "active_page": "signup"},
    )


@app.get("/health", summary="Simple health check")
def health() -> dict:
    """Health check that reflects DB connectivity.

    Attempts a short-lived connection to the configured database engine
    and returns 200 when healthy or 503 when the DB cannot be reached.
    """
    from sqlalchemy.exc import SQLAlchemyError

    try:
        # perform a quick connection test
        from app.db import engine

        with engine.connect() as conn:
            conn.exec_driver_sql("SELECT 1")
        return {"status": "ok"}
    except SQLAlchemyError:
        return PlainTextResponse(
            content="DB connectivity failure",
            status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
        )
    except Exception:
        return PlainTextResponse(
            content="Health check failed",
            status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
        )
```

models.py

```python
from __future__ import annotations

from datetime import datetime, timezone
from typing import Optional

from pydantic import EmailStr
from sqlmodel import Field, SQLModel


def utcnow() -> datetime:
    return datetime.now(timezone.utc)


class User(SQLModel, table=True):
    __tablename__ = "users"

    id: Optional[int] = Field(default=None, primary_key=True)
    email: EmailStr = Field(index=True, sa_column_kwargs={"unique": True})
    full_name: str = Field(default="")
    hashed_password: str
    created_at: datetime = Field(default_factory=utcnow)
    updated_at: datetime = Field(default_factory=utcnow)


class QRItem(SQLModel, table=True):
    __tablename__ = "qr_items"

    id: Optional[int] = Field(default=None, primary_key=True)
    user_id: int = Field(foreign_key="users.id", index=True)
    title: Optional[str] = Field(default=None, max_length=200)
    url: str
    foreground_color: str = Field(default="#000000", max_length=7)
    background_color: str = Field(default="#FFFFFF", max_length=7)
    size: int = Field(default=256, ge=64, le=1024)
    padding: int = Field(default=10, ge=0, le=80)
    border_radius: int = Field(default=0, ge=0, le=60)
    overlay_text: Optional[str] = Field(default=None, max_length=4)
    svg_path: str
    png_path: Optional[str] = Field(default=None)
    created_at: datetime = Field(default_factory=utcnow, index=True)
    updated_at: datetime = Field(default_factory=utcnow)
```

schemas.py

```python
from datetime import datetime
from typing import Annotated, Optional
```

```
from pydantic import BaseModel, ConfigDict, EmailStr, Field, HttpUrl

HexColor = Annotated[str, Field(pattern=r"^#[0-9a-fA-F]{6}$")]
HexOrTransparent = Annotated[str, Field(pattern=r"^(#[0-9a-fA-F]
{6}|transparent)$")]


class UserBase(BaseModel):
    email: EmailStr
    full_name: Optional[str] = None


class UserCreate(UserBase):
    password: str = Field(min_length=8)

    model_config = ConfigDict(
        json_schema_extra={
            "example": {
                "email": "user@example.com",
                "full_name": "Sample User",
                "password": "strongpass123",
            }
        }
    )


class UserRead(UserBase):
    id: int
    created_at: datetime
    updated_at: datetime

    model_config = ConfigDict(
        json_schema_extra={
            "example": {
                "id": 1,
                "email": "user@example.com",
                "full_name": "Sample User",
                "created_at": "2024-01-01T12:00:00Z",
                "updated_at": "2024-01-02T12:00:00Z",
            }
        }
    )


class UserLogin(BaseModel):
    email: EmailStr
    password: str

    model_config = ConfigDict(
        json_schema_extra={
            "example": {
                "email": "user@example.com",
                "password": "strongpass123",
            }
        }
    )
```

```python
class UserUpdate(BaseModel):
    full_name: Optional[str] = None
    password: Optional[str] = Field(default=None, min_length=8)


class Token(BaseModel):
    access_token: str
    token_type: str = "bearer"

    model_config = ConfigDict(
        json_schema_extra={
            "example": {
                "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
                "token_type": "bearer",
            }
        }
    )


class TokenPayload(BaseModel):
    sub: int
    exp: datetime


class QRBase(BaseModel):
    title: str = Field(max_length=200)
    url: HttpUrl
    foreground_color: HexColor = "#000000"
    background_color: HexOrTransparent = "#FFFFFF"
    size: int = Field(default=512, ge=128, le=1024)
    padding: int = Field(default=16, ge=0, le=128)
    border_radius: int = Field(default=0, ge=0, le=120)
    overlay_text: Optional[str] = Field(default=None, max_length=4)

    model_config = ConfigDict(
        json_schema_extra={
            "example": {
                "title": "Campaign QR",
                "url": "https://example.com",
                "foreground_color": "#1f3a93",
                "background_color": "#ffffff",
                "size": 320,
                "padding": 12,
                "border_radius": 20,
            }
        }
    )


class QRCreate(QRBase):
    pass


class QRPreviewResponse(BaseModel):
    svg_data: str
```

```
    png_data: str

    model_config = ConfigDict(
        json_schema_extra={
            "example": {
                "svg_data": "<svg xmlns=...>",
                "png_data": "iVBORw0KGgoAAAANSUhEUg...",
            }
        }
    )
```

server.py

```
"""ASGI entrypoint that exposes the FastAPI `app` instance.

This separate module avoids package/module name collisions and keeps the
container entrypoint simple: `uvicorn app.server:app`.
"""

from __future__ import annotations

from fastapi import FastAPI

# Import the top-level app instance defined in `app/main.py`.
from app.main import app as application


app: FastAPI = application
```

storage.py

app\assets

app\assets\icons

app\core

init.py

bootstrap.py

```
"""Helpers that prepare runtime filesystem state for the application.

This module provides `ensure_dirs` which creates asset and temp directories
at application startup. Directory creation is tolerant to failures (useful
for CI runners with restricted permissions).
"""

from pathlib import Path
from typing import Iterable

from app.config import settings


def ensure_dirs(s=None) -> Iterable[Path]:
    """Ensure asset and temp directories exist. Returns created Path objects.

    If `s` is not provided, use module `settings`.
    """
    s = s or settings
    created = []
    for p in (s.assets_dir, s.temp_dir):
        try:
            Path(p).mkdir(parents=True, exist_ok=True)
            created.append(Path(p))
        except Exception:
            # In some CI/runners writing to the configured path may fail; let
            # the app continue and surface the error elsewhere if needed.
            pass
    return created
```

logging.py

```python
"""Centralised structured logging configuration for the application.

This module exposes `configure_logging` to initialise the root logger and
`get_logger` as a convenience wrapper.
"""

import logging
import os
from typing import Optional


def configure_logging(level: Optional[str] = None) -> None:
    """Configure application logging.

    Uses `LOG_LEVEL` env var as default (INFO).
    """
    log_level = (level or os.getenv("LOG_LEVEL", "INFO")).upper()
    numeric_level = getattr(logging, log_level, logging.INFO)

    fmt = "%(asctime)s %(levelname)s [%(name)s:%(module)s] %(message)s"
    datefmt = "%Y-%m-%dT%H:%M:%S%z"

    handler = logging.StreamHandler()
    handler.setFormatter(logging.Formatter(fmt=fmt, datefmt=datefmt))

    root = logging.getLogger()
    # avoid adding multiple handlers when called repeatedly
    if not root.handlers:
        root.addHandler(handler)
    root.setLevel(numeric_level)


def get_logger(name: str) -> logging.Logger:
    return logging.getLogger(name)
```

security.py

```python
from datetime import datetime, timedelta, timezone
from functools import lru_cache
from typing import Optional

import bcrypt
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from jose import JWTError, jwt
from sqlmodel import Session

from app.config import settings
from app.db import get_session
from app.models import User
```

```
http_bearer = HTTPBearer(auto_error=False)


class _AuthError(HTTPException):
    """Customised HTTP exception for authentication failures."""

    def __init__(self, detail: str) -> None:
        super().__init__(status_code=status.HTTP_401_UNAUTHORIZED, detail=detail)


def verify_password(plain_password: str, hashed_password: str) -> bool:
    """Return True when the provided password matches the stored bcrypt hash."""

    return bcrypt.checkpw(
        plain_password.encode("utf-8"), hashed_password.encode("utf-8")
    )


def get_password_hash(password: str) -> str:
    """Hash the provided password using bcrypt with a randomly generated salt."""

    return bcrypt.hashpw(password.encode("utf-8"), bcrypt.gensalt()).decode("utf-
8")


def create_access_token(
    *, subject: int, expires_delta: Optional[timedelta] = None
) -> str:
    """Create a signed JWT using the configured algorithm and expiry."""

    expire_delta = expires_delta or timedelta(
        minutes=settings.access_token_expire_minutes
    )
    expire = datetime.now(timezone.utc) + expire_delta
    payload = {"sub": str(subject), "exp": expire}
    return jwt.encode(payload, settings.secret_key, algorithm=settings.algorithm)


@lru_cache(maxsize=128)
def _decode_access_token(token: str) -> dict:
    """Decode and cache JWT payloads to avoid repeated signature checks in a
request."""

    return jwt.decode(token, settings.secret_key, algorithms=[settings.algorithm])


async def get_current_user(
    credentials: Optional[HTTPAuthorizationCredentials] = Depends(http_bearer),
    session: Session = Depends(get_session),
) -> User:
    """Retrieve the authenticated user based on the Authorization header."""

    if not credentials:
        raise _AuthError("Not authenticated")

    token = credentials.credentials
```

```
    try:
        payload = _decode_access_token(token)
        subject = payload.get("sub")
        if subject is None:
            raise _AuthError("Invalid token payload")
        user_id = int(subject)
    except (JWTError, ValueError):
        raise _AuthError("Invalid token") from None

    user = session.get(User, user_id)
    if not user:
        raise _AuthError("User not found")

    return user
```

📁 app\generated_pngs

📁 app\generated_svgs

📁 app\home

📄 init.py

📁 app\home\site

📁 app\home\site\wwwroot

📄 qrcodes.db

⚠️ Could not read file: 'utf-8' codec can't decode byte 0x86 in position 98: invalid start byte

app\monitoring

init.py

docker-compose.monitoring.yml

```
version: '3.8'
services:
  app:
    image: qr-app:latest
    ports:
      - '8000:80'
    environment:
      - PORT=80
  prometheus:
    image: prom/prometheus:latest
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
    ports:
      - '9090:9090'
  grafana:
    image: grafana/grafana:latest
    ports:
      - '3000:3000'
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=admin
      - GF_INSTALL_PLUGINS=grafana-piechart-panel
    depends_on:
      - prometheus
```

grafana-dashboard.json

```
{
  "title": "QR Forge Overview",
  "panels": [
    {"type": "stat", "title": "Uptime", "targets": []},
```

```
    {"type": "graph", "title": "Request rate (rate)", "targets": []},
    {"type": "graph", "title": "P95 latency", "targets": []},
    {"type": "graph", "title": "P99 latency", "targets": []},
    {"type": "graph", "title": "4xx/5xx rate", "targets": []},
    {"type": "graph", "title": "DB connections", "targets": []}
  ],
  "time": {"from": "now-1h", "to": "now"}
}
{
  "title": "QR Forge - Basic Metrics",
  "panels": [
    {
      "type": "graph",
      "title": "Request Count",
      "targets": [
        { "expr": "sum by (endpoint)(increase(qr_forge_requests_total[5m]))",
"legendFormat": "{{endpoint}}" }
      ]
    },
    {
      "type": "graph",
      "title": "Average Request Latency (s)",
      "targets": [
        { "expr": "avg by (endpoint)(rate(qr_forge_request_latency_seconds_sum[5m])
/ rate(qr_forge_request_latency_seconds_count[5m]))", "legendFormat": "
{{endpoint}}" }
      ]
    },
    {
      "type": "graph",
      "title": "5xx Error Rate",
      "targets": [
        { "expr": "sum(increase(qr_forge_requests_total{http_status=~\"5..\"}[5m]))
/ sum(increase(qr_forge_requests_total[5m]))", "legendFormat": "5xx rate" }
      ]
    }
  ]
}
```

prometheus.yml

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: 'qr_forge_app'
    metrics_path: /metrics
    scheme: http
    static_configs:
      - targets: ['qr-forge:8000']
```

```
  - job_name: 'azure_webapp'
    kubernetes_sd_configs: []
    metrics_path: /metrics
    relabel_configs: []
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: 'qr_forge_app'
    static_configs:
      - targets: ['app:8000']
        labels:
          group: 'qr_forge'
    metrics_path: /metrics
```

📁 app\routers

📄 init.py

```
from . import auth, export, qr, user

__all__ = ["auth", "export", "qr", "user"]
```

📄 auth.py

```
from fastapi import APIRouter, Depends, status
from sqlmodel import Session

from app.db import get_session
from app.models import User
from app.schemas import Token, UserCreate, UserLogin, UserRead
from app.services.auth import login_user, signup_user

router = APIRouter(prefix="/api/auth", tags=["auth"])


@router.post(
    "/signup",
    response_model=UserRead,
    status_code=status.HTTP_201_CREATED,
    summary="Create a new user account",
```

```
    response_description="Newly created user profile",
)
def signup(payload: UserCreate, session: Session = Depends(get_session)) -> User:
    return signup_user(session, payload)


@router.post(
    "/login",
    response_model=Token,
    summary="Authenticate and receive an access token",
    response_description="Bearer token for subsequent requests",
)
def login(payload: UserLogin, session: Session = Depends(get_session)) -> Token:
    return login_user(session, payload)


@router.post("/logout", summary="Client-side logout acknowledgement")
def logout() -> dict:
    return {"ok": True}
```

export.py

```
import csv
import io
from pathlib import Path
from typing import Iterable

from fastapi import APIRouter, Depends
from fastapi.responses import StreamingResponse
from sqlmodel import Session, select

from app.core.security import get_current_user
from app.db import get_session
from app.models import QRItem, User

router = APIRouter(prefix="/api/export", tags=["export"])


@router.get(
    "/csv",
    summary="Export the authenticated user's QR history as CSV",
    response_description="CSV stream containing saved QR metadata",
)
def export_csv(
    session: Session = Depends(get_session),
    current_user: User = Depends(get_current_user),
) -> StreamingResponse:
    rows = session.exec(
        select(QRItem)
        .where(QRItem.user_id == current_user.id)
        .order_by(QRItem.created_at.desc())
```

```
    ).all()

    buf = io.StringIO()
    writer = csv.writer(buf)
    writer.writerow(
        [
            "title",
            "url",
            "created_at",
            "foreground_color",
            "background_color",
            "size",
            "padding",
            "border_radius",
            "svg_file",
            "png_file",
        ]
    )
    for r in rows:
        writer.writerow(
            [
                r.title,
                r.url,
                r.created_at.isoformat(),
                r.foreground_color,
                r.background_color,
                r.size,
                r.padding,
                r.border_radius,
                Path(r.svg_path).name if r.svg_path else "",
                Path(r.png_path).name if r.png_path else "",
            ]
        )
    buf.seek(0)

    def iter_buf() -> Iterable[str]:
        yield buf.read()

    return StreamingResponse(
        iter_buf(),
        media_type="text/csv",
        headers={"Content-Disposition": "attachment; filename=qr_history.csv"},
    )
```

qr.py

```
from pathlib import Path
from typing import List

from fastapi import APIRouter, Depends, Query, status
from fastapi.responses import FileResponse
```

```python
from sqlmodel import Session

from app.config import settings
from app.core.security import get_current_user
from app.db import get_session
from app.models import QRItem, User
from app.schemas import QRCreate, QRPreviewResponse
from app.services import qr_items

router = APIRouter(prefix="/api/qr", tags=["qr"])
SVG_DIR = settings.assets_dir
PNG_DIR = settings.temp_dir


@router.post(
    "/preview",
    response_model=QRPreviewResponse,
    summary="Render a customised QR preview without saving",
    response_description="Inline base64 PNG and SVG markup",
)
def preview_qr(
    payload: QRCreate,
    current_user: User = Depends(get_current_user),
) -> QRPreviewResponse:
    _ = current_user
    return qr_items.preview_qr(payload)


@router.post(
    "",
    response_model=QRItem,
    status_code=status.HTTP_201_CREATED,
    summary="Persist a customised QR code",
    response_description="Saved QR item with asset paths",
)
def create_qr(
    payload: QRCreate,
    session: Session = Depends(get_session),
    current_user: User = Depends(get_current_user),
) -> QRItem:
    return qr_items.create_qr_item(
        session,
        payload,
        current_user,
        svg_dir=SVG_DIR,
        png_dir=PNG_DIR,
    )


@router.get(
    "",
    response_model=List[QRItem],
    summary="List QR codes owned by the authenticated user",
)
def list_qr(
    session: Session = Depends(get_session),
    current_user: User = Depends(get_current_user),
```

```
) -> List[QRItem]:
    return qr_items.list_qr_items(session, current_user)


@router.get(
    "/history",
    response_model=List[QRItem],
    summary="Alias for listing QR history",
)
def history(
    session: Session = Depends(get_session),
    current_user: User = Depends(get_current_user),
) -> List[QRItem]:
    return qr_items.list_qr_items(session, current_user)


@router.delete(
    "/{item_id}",
    summary="Delete a saved QR code",
    response_description="Confirmation payload",
)
def delete_qr(
    item_id: int,
    session: Session = Depends(get_session),
    current_user: User = Depends(get_current_user),
) -> dict:
    qr_items.delete_qr_item(session, current_user, item_id)
    return {"ok": True}


@router.get(
    "/{item_id}/download",
    summary="Download a saved QR code",
    response_description="Binary SVG or PNG stream",
)
def download_qr(
    item_id: int,
    format: str = Query(default="svg", pattern="^(svg|png)$"),
    session: Session = Depends(get_session),
    current_user: User = Depends(get_current_user),
):
    path: Path = qr_items.download_path(session, current_user, item_id, format)
    media_type = "image/svg+xml" if format == "svg" else "image/png"
    return FileResponse(path, media_type=media_type, filename=f"qr-{item_id}.
{format}")
```

user.py

```
from fastapi import APIRouter, Depends
from sqlmodel import Session
```

```python
from app.core.security import get_current_user
from app.db import get_session
from app.models import User
from app.schemas import UserRead, UserUpdate
from app.services.user import delete_user_and_assets, update_profile

router = APIRouter(prefix="/api/user", tags=["users"])


@router.get(
    "/me",
    response_model=UserRead,
    summary="Return the authenticated user's profile",
    response_description="Current user record",
)
def read_current_user(current_user: User = Depends(get_current_user)) -> User:
    return current_user


@router.patch(
    "/me",
    response_model=UserRead,
    summary="Update profile details",
    response_description="Updated user record",
)
def update_current_user(
    payload: UserUpdate,
    session: Session = Depends(get_session),
    current_user: User = Depends(get_current_user),
) -> User:
    return update_profile(session, current_user, payload)


@router.delete(
    "/me",
    summary="Delete the authenticated user and all owned QR codes",
    response_description="Confirmation payload",
)
def delete_current_user(
    session: Session = Depends(get_session),
    current_user: User = Depends(get_current_user),
) -> dict:
    delete_user_and_assets(session, current_user)
    return {"ok": True}
```

📂 app\services

📄 init.py

auth.py

```
from __future__ import annotations

from datetime import datetime, timezone

from fastapi import HTTPException, status
from sqlmodel import Session, select

from app.core.security import (
    create_access_token,
    get_password_hash,
    verify_password,
)
from app.models import User
from app.schemas import Token, UserCreate, UserLogin


def _normalize_email(email: str) -> str:
    return email.lower()


def signup_user(session: Session, payload: UserCreate) -> User:
    normalized_email = _normalize_email(payload.email)
    existing = session.exec(select(User).where(User.email ==
normalized_email)).first()
    if existing:
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT, detail="Email already registered"
        )

    now = datetime.now(timezone.utc)
    user = User(
        email=normalized_email,
        full_name=payload.full_name or "",
        hashed_password=get_password_hash(payload.password),
        created_at=now,
        updated_at=now,
    )
    session.add(user)
    session.commit()
    session.refresh(user)
    return user


def login_user(session: Session, payload: UserLogin) -> Token:
    normalized_email = _normalize_email(payload.email)
    user = session.exec(select(User).where(User.email == normalized_email)).first()
    if not user or not verify_password(payload.password, user.hashed_password):
```

```
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid credentials"
        )

    token = create_access_token(subject=user.id)
    return Token(access_token=token)
```

qr.py

```
from __future__ import annotations

import base64
import io
import uuid
from dataclasses import dataclass
from pathlib import Path
from typing import List, Tuple

import qrcode
from PIL import Image, ImageDraw


@dataclass
class QRConfig:
    url: str
    foreground_color: str
    background_color: str
    size: int
    padding: int
    border_radius: int


@dataclass
class QRPreview:
    svg_data: str
    png_data: str


@dataclass
class QRRender:
    svg_text: str
    png_bytes: bytes


@dataclass
class QRAssets:
    svg_path: Path
    png_path: Path


HEX_ALPHA = 255
```

```
    TRANSPARENT = (0, 0, 0, 0)


    def _ensure_dir(path: Path) -> Path:
        path.mkdir(parents=True, exist_ok=True)
        return path


    def _hex_to_rgba(color: str) -> Tuple[int, int, int, int]:
        if color.lower() == "transparent":
            return TRANSPARENT
        color = color.lstrip("#")
        if len(color) != 6:
            raise ValueError('Expected 6 character hex color or "transparent"')
        r = int(color[0:2], 16)
        g = int(color[2:4], 16)
        b = int(color[4:6], 16)
        return r, g, b, HEX_ALPHA


    def _create_matrix(config: QRConfig) -> List[List[bool]]:
        qr = qrcode.QRCode(
            version=None,
            error_correction=qrcode.constants.ERROR_CORRECT_M,
            border=0,
        )
        qr.add_data(config.url)
        qr.make(fit=True)
        return qr.get_matrix()


    def _render_svg(config: QRConfig, matrix: List[List[bool]]) -> str:
        modules = len(matrix)
        module_size = config.size / modules
        total_size = config.size + config.padding * 2
        bg = config.background_color
        svg_parts = [
            (
                f'<svg xmlns="http://www.w3.org/2000/svg" '
                f'width="{total_size}" height="{total_size}" '
                f'viewBox="0 0 {total_size} {total_size}">'
            )
        ]
        if bg.lower() != "transparent":
            svg_parts.append(
                (
                    f'<rect width="{total_size}" height="{total_size}" '
                    f'fill="{bg}" rx="{config.border_radius}" '
                    f'ry="{config.border_radius}" />'
                )
            )
        pad = config.padding
        fg = config.foreground_color
        for y, row in enumerate(matrix):
            for x, cell in enumerate(row):
                if not cell:
                    continue
```

```
            x0 = pad + x * module_size
            y0 = pad + y * module_size
            svg_parts.append(
                (
                    f'<rect x="{x0:.3f}" y="{y0:.3f}" '
                    f'width="{module_size:.3f}" height="{module_size:.3f}" '
                    f'fill="{fg}" />'
                )
            )
    svg_parts.append("</svg>")
    return "".join(svg_parts)


def _render_png(config: QRConfig, matrix: List[List[bool]]) -> bytes:
    modules = len(matrix)
    module_size = config.size / modules
    total_size = config.size + config.padding * 2

    background = Image.new(
        "RGBA", (total_size, total_size), _hex_to_rgba(config.background_color)
    )
    draw = ImageDraw.Draw(background)
    fg_rgba = _hex_to_rgba(config.foreground_color)

    for y, row in enumerate(matrix):
        for x, cell in enumerate(row):
            if not cell:
                continue
            x0 = config.padding + x * module_size
            y0 = config.padding + y * module_size
            x1 = x0 + module_size
            y1 = y0 + module_size
            draw.rectangle([x0, y0, x1, y1], fill=fg_rgba)

    if config.border_radius > 0:
        radius = min(config.border_radius, total_size // 2)
        mask = Image.new("L", (total_size, total_size), 0)
        mask_draw = ImageDraw.Draw(mask)
        mask_draw.rounded_rectangle(
            (0, 0, total_size, total_size), radius=radius, fill=255
        )
        rounded = Image.new("RGBA", (total_size, total_size))
        rounded.paste(background, (0, 0), mask=mask)
        background = rounded

    with io.BytesIO() as buf:
        background.save(buf, format="PNG")
        return buf.getvalue()


def render_qr(config: QRConfig) -> QRRender:
    matrix = _create_matrix(config)
    svg_text = _render_svg(config, matrix)
    png_bytes = _render_png(config, matrix)
    return QRRender(svg_text=svg_text, png_bytes=png_bytes)
```

```python
def generate_qr_assets(
    config: QRConfig,
    *,
    svg_dir: Path,
    png_dir: Path,
) -> QRAssets:
    svg_dir = _ensure_dir(svg_dir)
    png_dir = _ensure_dir(png_dir)
    render = render_qr(config)

    svg_filename = f"{uuid.uuid4()}.svg"
    svg_path = svg_dir / svg_filename
    svg_path.write_text(render.svg_text, encoding="utf-8")

    png_filename = f"{svg_filename[:-4]}.png"
    png_path = png_dir / png_filename
    png_path.write_bytes(render.png_bytes)

    return QRAssets(svg_path=svg_path, png_path=png_path)


def encode_render(render: QRRender) -> QRPreview:
    return QRPreview(
        svg_data=render.svg_text,
        png_data=base64.b64encode(render.png_bytes).decode("ascii"),
    )
```

qr_items.py

```python
from __future__ import annotations

from datetime import datetime, timezone
from pathlib import Path
from typing import List

from fastapi import HTTPException, status
from sqlmodel import Session, select

from app.models import QRItem, User
from app.schemas import QRCreate, QRPreviewResponse
from app.services.qr import (
    QRConfig,
    QRPreview,
    encode_render,
    generate_qr_assets,
    render_qr,
)


def build_config(payload: QRCreate) -> QRConfig:
    return QRConfig(
```

```
        url=str(payload.url),
        foreground_color=payload.foreground_color,
        background_color=payload.background_color,
        size=payload.size,
        padding=payload.padding,
        border_radius=payload.border_radius,
    )


def preview_qr(payload: QRCreate) -> QRPreviewResponse:
    preview: QRPreview = encode_render(render_qr(build_config(payload)))
    return QRPreviewResponse(svg_data=preview.svg_data, png_data=preview.png_data)


def create_qr_item(
    session: Session,
    payload: QRCreate,
    current_user: User,
    *,
    svg_dir: Path,
    png_dir: Path,
) -> QRItem:
    now = datetime.now(timezone.utc)
    assets = generate_qr_assets(build_config(payload), svg_dir=svg_dir,
png_dir=png_dir)

    item = QRItem(
        user_id=current_user.id,
        title=payload.title.strip() or "Untitled QR",
        url=str(payload.url),
        foreground_color=payload.foreground_color,
        background_color=payload.background_color,
        size=payload.size,
        padding=payload.padding,
        border_radius=payload.border_radius,
        overlay_text=payload.overlay_text,
        svg_path=str(assets.svg_path),
        png_path=str(assets.png_path),
        created_at=now,
        updated_at=now,
    )
    session.add(item)
    session.commit()
    session.refresh(item)
    return item


def list_qr_items(session: Session, current_user: User) -> List[QRItem]:
    return session.exec(
        select(QRItem)
        .where(QRItem.user_id == current_user.id)
        .order_by(QRItem.created_at.desc())
    ).all()


def get_owned_item(session: Session, current_user: User, item_id: int) -> QRItem:
    item = session.exec(
```

```
        select(QRItem).where(QRItem.id == item_id, QRItem.user_id ==
current_user.id)
    ).first()
    if not item:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND, detail="QR item not found"
        )
    return item


def delete_qr_item(session: Session, current_user: User, item_id: int) -> None:
    item = get_owned_item(session, current_user, item_id)
    _delete_if_exists(item.svg_path)
    _delete_if_exists(item.png_path)
    session.delete(item)
    session.commit()


def download_path(
    session: Session, current_user: User, item_id: int, format: str
) -> Path:
    item = get_owned_item(session, current_user, item_id)
    if format == "svg":
        path = Path(item.svg_path)
        if not path.exists():
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND, detail="SVG not available"
            )
        return path

    if not item.png_path:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="PNG export not available yet",
        )
    path = Path(item.png_path)
    if not path.exists():
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND, detail="PNG not available"
        )
    return path


def _delete_if_exists(path: str | None) -> None:
    if not path:
        return
    p = Path(path)
    if p.exists():
        p.unlink()
```

user.py

```python
from __future__ import annotations

from datetime import datetime, timezone
from pathlib import Path

from sqlmodel import Session, select

from app.core.security import get_password_hash
from app.models import QRItem, User
from app.schemas import UserUpdate


def update_profile(session: Session, current_user: User, payload: UserUpdate) ->
User:
    updated = False
    if payload.full_name is not None:
        current_user.full_name = payload.full_name.strip()
        updated = True
    if payload.password:
        current_user.hashed_password = get_password_hash(payload.password)
        updated = True

    if not updated:
        return current_user

    current_user.updated_at = datetime.now(timezone.utc)
    session.add(current_user)
    session.commit()
    session.refresh(current_user)
    return current_user


def delete_user_and_assets(session: Session, current_user: User) -> None:
    qrs = session.exec(select(QRItem).where(QRItem.user_id ==
current_user.id)).all()
    for item in qrs:
        _delete_asset_file(item.svg_path)
        _delete_asset_file(item.png_path)
        session.delete(item)
    session.delete(current_user)
    session.commit()


def _delete_asset_file(path: str | None) -> None:
    if not path:
        return
    p = Path(path)
    if p.exists():
        p.unlink()
```

📁 app\static

app\templates

base.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <title>{% block title %}QR Forge{% endblock %}</title>
  <meta name="viewport" content="width=device-width, initial-scale=1" />

  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?
family=Mulish:ital,wght@0,200..1000;1,200..1000&display=swap" rel="stylesheet">

  <link rel="stylesheet" href="/static/styles.css" />
  {% block head_scripts %}{% endblock %}
</head>
<body class="page-{{ active_page|default('home') }}">
  <div class="bg-shape s1"></div>
  <div class="bg-shape s2"></div>
  <div class="bg-shape s3"></div>

  <div class="wrap">
    <aside class="toolbar card">
      <div class="logo">QF</div>
      <nav class="toolbar-nav">
        <a id="navHome" href="{{ url_for('home') }}" class="tool{% if active_page
== 'home' %} active{% endif %}" title="Home">
          <img src="/assets/icons/home.svg" alt="" class="tool-icon" />
          <span class="tool-label">Home</span>
        </a>
        <a id="navGenerator" href="{{ url_for('generator_page') }}" class="tool{%
if active_page == 'generator' %} active{% endif %}" title="Generator">
          <img src="/assets/icons/generator.svg" alt="" class="tool-icon" />
          <span class="tool-label">Generator</span>
        </a>
        <a id="navHistory" href="{{ url_for('history_page') }}" class="tool{% if
active_page == 'history' %} active{% endif %}" title="History">
          <img src="/assets/icons/history.svg" alt="" class="tool-icon" />
          <span class="tool-label">History</span>
        </a>
        <a id="navProfile" href="{{ url_for('profile_page') }}" class="tool{% if
active_page == 'profile' %} active{% endif %}" title="Profile" data-requires-auth>
          <img src="/assets/icons/profile.svg" alt="" class="tool-icon" />
          <span class="tool-label">Profile</span>
        </a>
        <a id="navLogin" href="{{ url_for('login_page') }}" class="tool{% if
active_page == 'login' %} active{% endif %}" title="Login" data-hide-when-authed>
          <img src="/assets/icons/login.svg" alt="" class="tool-icon" />
          <span class="tool-label">Login</span>
```

```
        </a>
      </nav>
    </aside>

    <main class="panel card" id="main-panel">
      {% block content %}{% endblock %}
    </main>
  </div>

  <div id="toast" class="toast" role="status" aria-live="polite"></div>
  {% block body_scripts %}{% endblock %}
  <script defer src="/static/app.js"></script>
</body>
</html>
```

history.html

```
{% extends "base.html" %}
{% block title %}History · QR Forge{% endblock %}

{% block content %}
<section class="history-panel">
  <div id="historyGuard" class="auth-gate">
    <h2>Sign in to view your QR history</h2>
    <p>Login to retrieve saved QR codes, download exports, and manage your library.
</p>
    <div class="gate-actions">
      <a class="btn primary" href="{{ url_for('login_page') }}">Login</a>
      <a class="btn ghost" href="{{ url_for('signup_page') }}">Create account</a>
    </div>
  </div>

  <div id="historyContent" class="history-content" data-history-auth>
    <header class="history-header">
      <h1 class="title">QR history</h1>
      <p class="subtitle">All of your generated QR codes, ready to download again
or remove.</p>
    </header>

    <div class="history-actions">
      <button id="refreshHistory" class="btn ghost" type="button">Refresh</button>
      <button id="exportCsv" class="btn" type="button">Export CSV</button>
    </div>

    <div id="historyEmpty" class="history-empty hidden">No QR codes yet. Generate
your first one from the generator tab.</div>
    <div id="historyGrid" class="history-grid"></div>
  </div>
</section>
```

```
{% endblock %}
```

home.html

```
{% extends "base.html" %}
{% block title %}Home · QR Forge{% endblock %}

{% block content %}
<section class="home-hero">
  <h1 class="title">Design. Personalize. Share.</h1>
  <p class="subtitle">QR Forge lets you craft branded QR codes with color, padding,
and overlays in seconds.</p>
  <div class="hero-actions">
    <a class="btn primary" href="{{ url_for('generator_page') }}">Start
generating</a>
    <a class="btn ghost" href="{{ url_for('history_page') }}">View history</a>
  </div>
</section>
{% endblock %}
```

index.html

```
{% extends "base.html" %}
{% block title %}Generator · QR Forge{% endblock %}

{% block content %}
<section class="left">
  <div id="generatorGuard" class="auth-gate">
    <h2>Sign in to generate QR codes</h2>
    <p>Create an account to store your designs, customize styles, and download SVG
or PNG exports.</p>
    <div class="gate-actions">
      <a class="btn primary" href="{{ url_for('login_page') }}">Login</a>
      <a class="btn ghost" href="{{ url_for('signup_page') }}">Create account</a>
    </div>
  </div>

  <div id="generatorFormWrap" data-generator-auth>
    <h1 class="title">Enter your text</h1>
    <p class="subtitle">Preview your QR code and save it when you're ready.</p>

    <form id="qr-form" class="input-card">
      <input id="url" name="url" type="url" placeholder="https://example.com"
required />
      <div class="row">
```

```
        <input id="title" name="title" type="text" placeholder="QR title
(optional)" />
        <button id="previewBtn" type="submit" class="btn primary">Preview</button>
      </div>
      <div class="controls" id="controls">
        <div class="control">
          <label>Foreground color</label>
          <input id="fgColor" type="color" value="#0a0a0a" />
        </div>
        <div class="control">
          <label>Background color</label>
          <div class="seg-2">
            <input id="bgColor" type="color" value="#ffffff" />
            <label class="checkbox-inline">
              <input id="bgTransparent" type="checkbox" /> Transparent
            </label>
          </div>
        </div>
        <div class="control">
          <label>Size <span id="sizeValue" class="control-value">512 px</span>
</label>
          <input id="sizeRange" type="range" min="128" max="1024" step="64"
value="512" />
        </div>
        <div class="control">
          <label>Padding <span id="paddingValue" class="control-value">16 px</span>
</label>
          <input id="paddingRange" type="range" min="0" max="80" step="4"
value="16" />
        </div>
        <div class="control">
          <label>Border radius</label>
          <input id="radiusRange" type="range" min="0" max="60" value="20" />
        </div>
      </div>
      <button id="saveQr" class="btn" type="button" disabled>Save to
history</button>
    </form>
  </div>
</section>

<section class="right" data-generator-auth>
  <div class="preview card">
    <div class="qr-box" id="qrBox">
      <img id="qrPreview" alt="QR preview" />
      <div id="qrEmpty" class="qr-empty">Your QR preview will appear here</div>
    </div>
    <div class="download-row">
      <button id="dlSvg" class="btn" type="button">Download SVG</button>
      <button id="dlPng" class="btn accent" type="button">Download PNG</button>
      <button id="openHistory" class="btn ghost" type="button">History</button>
    </div>
  </div>
</section>

<aside id="historyDrawer" class="history-drawer" data-generator-auth>
  <div class="drawer-head">
```

```
    <h3>Recent QRs</h3>
    <button id="closeHistory" class="btn ghost small" type="button">Close</button>
  </div>
  <div id="historyList" class="history-list"></div>
</aside>
{% endblock %}
```

login.html

```
{% extends "base.html" %}
{% block title %}Login · QR Forge{% endblock %}

{% block content %}
<section class="auth-panel">
  <div class="auth-copy">
    <h1 class="title">Welcome back</h1>
    <p class="subtitle">Sign in to manage your QR history and profile.</p>
  </div>
  <form id="login-form" class="auth-card">
    <label class="form-field">
      <span>Email</span>
      <input type="email" name="email" placeholder="you@example.com" required />
    </label>
    <label class="form-field">
      <span>Password</span>
      <input type="password" name="password" placeholder="••••••••" required />
    </label>
    <button type="submit" class="btn primary full">Login</button>
    <p class="auth-link">
      No account yet?
      <a href="{{ url_for('signup_page') }}">Create one</a>
    </p>
  </form>
</section>
{% endblock %}
```

profile.html

```
{% extends "base.html" %}
{% block title %}Profile · QR Forge{% endblock %}

{% block content %}
<section class="profile-panel">
  <div id="profileGuard" class="auth-gate">
    <h2>You are not signed in</h2>
```

```
    <p>Login to edit your profile, review account details, and manage security
settings.</p>
    <div class="gate-actions">
      <a class="btn primary" href="{{ url_for('login_page') }}">Login</a>
      <a class="btn ghost" href="{{ url_for('signup_page') }}">Create account</a>
    </div>
  </div>

  <div id="profileContent" class="profile-details hidden" data-profile-auth>
    <header class="profile-header">
      <h1 class="title">Your profile</h1>
      <p class="subtitle">Manage your personal details and security preferences.
</p>
    </header>

    <section class="profile-card">
      <h2>Account</h2>
      <form id="profileForm" class="profile-form">
        <label class="form-field">
          <span>Full name</span>
          <input id="profileFullName" type="text" name="full_name"
placeholder="Your name" />
        </label>
        <label class="form-field">
          <span>Email</span>
          <input id="profileEmail" type="email" disabled />
        </label>
        <label class="form-field">
          <span>Member since</span>
          <input id="profileSince" type="text" disabled />
        </label>
        <label class="form-field">
          <span>New password (optional)</span>
          <input id="profilePassword" type="password" name="password"
placeholder="••••••••" />
        </label>
        <div class="profile-actions">
          <button class="btn primary" id="saveProfile" type="submit">Save
changes</button>
          <button class="btn" id="logoutBtn" type="button">Logout</button>
          <button class="btn danger" id="deleteAccount" type="button">Delete
account</button>
        </div>
      </form>
    </section>

    <section class="profile-card">
      <h2>Security</h2>
      <p class="subtitle">Keep your account safe by using a strong password.
Password updates are currently handled above.</p>
    </section>
  </div>
</section>
{% endblock %}
```

signup.html

```
{% extends "base.html" %}
{% block title %}Sign Up · QR Forge{% endblock %}

{% block content %}
<section class="auth-panel">
  <div class="auth-copy">
    <h1 class="title">Create your account</h1>
    <p class="subtitle">Track your QR creations across devices.</p>
  </div>
  <form id="signup-form" class="auth-card">
    <label class="form-field">
      <span>Full name</span>
      <input type="text" name="full_name" placeholder="Ada Lovelace" required />
    </label>
    <label class="form-field">
      <span>Email</span>
      <input type="email" name="email" placeholder="you@example.com" required />
    </label>
    <label class="form-field">
      <span>Password</span>
      <input type="password" name="password" placeholder="At least 8 characters"
required />
    </label>
    <button type="submit" class="btn primary full">Create account</button>
    <p class="auth-link">
      Already have an account?
      <a href="{{ url_for('login_page') }}">Login</a>
    </p>
  </form>
</section>
{% endblock %}
```

Assignment1

Assignment1\report

Assignment1\report\annex

Assignment1\report\annex\wireframe-mockup

Assignment2

logs

tests

conftest.py

```
import sys
from pathlib import Path
from typing import Generator

ROOT_DIR = Path(__file__).resolve().parents[1]
if str(ROOT_DIR) not in sys.path:
    sys.path.append(str(ROOT_DIR))

import pytest  # noqa: E402
from fastapi.testclient import TestClient  # noqa: E402
from sqlalchemy.pool import StaticPool  # noqa: E402
from sqlmodel import Session, SQLModel, create_engine  # noqa: E402

from app.main import app  # noqa: E402
from app.db import get_session  # noqa: E402

TEST_DATABASE_URL = "sqlite://"


def _create_engine():
    return create_engine(
        TEST_DATABASE_URL,
        connect_args={"check_same_thread": False},
        poolclass=StaticPool,
    )


@pytest.fixture(scope="session")
def engine() -> Generator:
    engine = _create_engine()
```

```
    yield engine


@pytest.fixture(autouse=True)
def prepare_database(engine) -> Generator:
    SQLModel.metadata.drop_all(engine)
    SQLModel.metadata.create_all(engine)
    yield
    SQLModel.metadata.drop_all(engine)


@pytest.fixture()
def client(tmp_path: Path, monkeypatch, engine) -> TestClient:
    def override_get_session() -> Generator[Session, None, None]:
        with Session(engine) as session:
            yield session

    app.dependency_overrides[get_session] = override_get_session

    from app.routers import qr

    svg_dir = tmp_path / "svg"
    png_dir = tmp_path / "png"
    svg_dir.mkdir(parents=True, exist_ok=True)
    png_dir.mkdir(parents=True, exist_ok=True)
    monkeypatch.setattr(qr, "SVG_DIR", svg_dir, raising=False)
    monkeypatch.setattr(qr, "PNG_DIR", png_dir, raising=False)

    return TestClient(app)
```

test_alembic_import.py

```
"""
Ensure Alembic environment scripts import without error
(offline/online compatibility).
"""


def test_alembic_env_importable():
    """Load the project's `alembic/env.py` by path to ensure it imports.

    Importing via the package name may resolve to the installed `alembic`
    distribution instead of the repository's migration scripts. Load by
    file path to guarantee we exercise the local `alembic/env.py`.
    """
    from pathlib import Path

    env_path = Path(__file__).resolve().parent.parent / "alembic" / "env.py"
    if not env_path.exists():
        raise AssertionError("alembic/env.py not found in repository")
```

```
    # Instead of executing top-level Alembic env (which expects the Alembic
    # runtime to provide `context.config`), validate the file for syntax
    # errors by parsing its AST. This verifies the migration script is well-
    # formed without requiring an Alembic runtime environment.
    import ast

    try:
        source = env_path.read_text(encoding="utf-8")
        ast.parse(source, filename=str(env_path))
    except Exception as exc:
        raise AssertionError(f"alembic/env.py is invalid Python: {exc}") from exc
```

test_auth.py

```
from fastapi.testclient import TestClient


def register_user(client: TestClient, email: str, password: str) -> None:
    payload = {
        "email": email,
        "full_name": "Test User",
        "password": password,
    }
    resp = client.post("/api/auth/signup", json=payload)
    assert resp.status_code == 201, resp.text


def authenticate(client: TestClient, email: str, password: str) -> dict:
    register_user(client, email, password)
    resp = client.post(
        "/api/auth/login",
        json={"email": email, "password": password},
    )
    assert resp.status_code == 200, resp.text
    return {"Authorization": f"Bearer {resp.json()['access_token']}"}


def test_signup_duplicate_email(client: TestClient) -> None:
    register_user(client, "dup@example.com", "password123")
    resp = client.post(
        "/api/auth/signup",
        json={
            "email": "dup@example.com",
            "full_name": "Another",
            "password": "password123",
        },
    )
    assert resp.status_code == 409
    assert "Email already registered" in resp.text
```

```python
def test_login_invalid_password(client: TestClient) -> None:
    register_user(client, "bob@example.com", "password123")
    resp = client.post(
        "/api/auth/login",
        json={"email": "bob@example.com", "password": "wrongpass"},
    )
    assert resp.status_code == 401
    assert "Invalid credentials" in resp.text


def test_logout_endpoint(client: TestClient) -> None:
    headers = authenticate(client, "logout@example.com", "password123")
    resp = client.post("/api/auth/logout", headers=headers)
    assert resp.status_code == 200
    assert resp.json()["ok"] is True


def test_login_missing_user(client: TestClient) -> None:
    resp = client.post(
        "/api/auth/login",
        json={"email": "ghost@example.com", "password": "password123"},
    )
    assert resp.status_code == 401
```

test_auth_and_qr.py

```python
import base64

from fastapi.testclient import TestClient
from sqlmodel import Session, select

from app.models import QRItem, User


def _auth_headers(
    client: TestClient, email: str = "alice@example.com", password: str =
"secret123"
) -> dict:
    signup_payload = {
        "email": email,
        "full_name": "Alice Example",
        "password": password,
    }
    resp = client.post("/api/auth/signup", json=signup_payload)
    assert resp.status_code == 201, resp.text

    login_payload = {"email": email, "password": password}
    resp = client.post("/api/auth/login", json=login_payload)
    assert resp.status_code == 200, resp.text
    token = resp.json()["access_token"]
    return {"Authorization": f"Bearer {token}"}
```

```python
def test_create_qr_requires_auth(client: TestClient) -> None:
    resp = client.post(
        "/api/qr",
        json={"title": "Unauth", "url": "https://example.com"},
    )
    assert resp.status_code == 401


def test_preview_requires_auth(client: TestClient) -> None:
    resp = client.post(
        "/api/qr/preview",
        json={
            "title": "No auth",
            "url": "https://example.com",
            "foreground_color": "#000000",
            "background_color": "#ffffff",
            "size": 256,
            "padding": 8,
            "border_radius": 8,
        },
    )
    assert resp.status_code == 401


def test_preview_and_lifecycle(client: TestClient) -> None:
    headers = _auth_headers(client)

    payload = {
        "title": "My QR",
        "url": "https://example.com",
        "foreground_color": "#123456",
        "background_color": "transparent",
        "size": 320,
        "padding": 12,
        "border_radius": 24,
    }
    preview_resp = client.post("/api/qr/preview", json=payload, headers=headers)
    assert preview_resp.status_code == 200, preview_resp.text
    preview = preview_resp.json()
    assert preview["png_data"]
    base64.b64decode(preview["png_data"])

    create_resp = client.post("/api/qr", json=payload, headers=headers)
    assert create_resp.status_code == 201, create_resp.text
    created = create_resp.json()
    assert created["background_color"] == "transparent"

    list_resp = client.get("/api/qr", headers=headers)
    assert list_resp.status_code == 200
    items = list_resp.json()
    assert len(items) == 1

    download_png = client.get(
        f"/api/qr/{created['id']}/download",
        params={"format": "png"},
```

```
        headers=headers,
    )
    assert download_png.status_code == 200
    assert download_png.headers["content-type"] == "image/png"

    delete_resp = client.delete(f"/api/qr/{created['id']}", headers=headers)
    assert delete_resp.status_code == 200
    assert delete_resp.json()["ok"] is True

    empty_resp = client.get("/api/qr", headers=headers)
    assert empty_resp.status_code == 200
    assert empty_resp.json() == []


def test_delete_user_removes_qrs(client: TestClient, engine) -> None:
    headers = _auth_headers(client)
    payload = {
        "title": "Keep",
        "url": "https://example.com",
        "foreground_color": "#000000",
        "background_color": "#ffffff",
        "size": 256,
        "padding": 8,
        "border_radius": 12,
    }
    resp = client.post("/api/qr", json=payload, headers=headers)
    assert resp.status_code == 201

    del_resp = client.delete("/api/user/me", headers=headers)
    assert del_resp.status_code == 200
    assert del_resp.json()["ok"] is True

    with Session(engine) as session:
        assert session.exec(select(User)).first() is None
        assert session.exec(select(QRItem)).all() == []


def test_update_profile_and_password_flow(client: TestClient) -> None:
    headers = _auth_headers(client, email="bob@example.com", password="initial123")

    update_resp = client.patch(
        "/api/user/me",
        json={"full_name": "Bob Builder", "password": "newsecret456"},
        headers=headers,
    )
    assert update_resp.status_code == 200, update_resp.text
    updated = update_resp.json()
    assert updated["full_name"] == "Bob Builder"

    me_resp = client.get("/api/user/me", headers=headers)
    assert me_resp.status_code == 200
    assert me_resp.json()["full_name"] == "Bob Builder"

    old_login = client.post(
        "/api/auth/login",
        json={"email": "bob@example.com", "password": "initial123"},
    )
```

```
    assert old_login.status_code == 401

    new_login = client.post(
        "/api/auth/login",
        json={"email": "bob@example.com", "password": "newsecret456"},
    )
    assert new_login.status_code == 200
    assert "access_token" in new_login.json()
```

test_auth_negative_extra.py

```
from fastapi.testclient import TestClient


def test_signup_invalid_email(client: TestClient) -> None:
    payload = {
        "email": "not-an-email",
        "full_name": "Bad",
        "password": "password123",
    }
    resp = client.post("/api/auth/signup", json=payload)
    assert resp.status_code == 422


def test_login_missing_password(client: TestClient) -> None:
    # Missing password field should return 422
    resp = client.post("/api/auth/login", json={"email": "a@b.com"})
    assert resp.status_code == 422
```

test_db_failure.py

```
import pytest

from app.db import get_engine
from app.config import settings


def test_get_engine_raises_when_no_url(monkeypatch) -> None:
    # If settings.database_url is empty or None and no database_url is
    # provided, get_engine should raise a RuntimeError.
    monkeypatch.setattr(settings, "database_url", "")
    with pytest.raises(RuntimeError):
        get_engine(None)
```

test_health_db_failure.py

```python
"""Tests for health endpoint reflecting DB connectivity failures."""

from fastapi.testclient import TestClient
from sqlalchemy.exc import SQLAlchemyError

from app.main import app as application


class BadEngine:
    def connect(self):
        raise SQLAlchemyError("connection failed")


def test_health_returns_503_on_db_failure(monkeypatch):
    """When the DB engine cannot connect, /health returns 503."""
    monkeypatch.setattr("app.db.engine", BadEngine())
    client = TestClient(application)
    resp = client.get("/health")
    assert resp.status_code == 503
    assert "DB connectivity" in resp.text or "Health check failed" in resp.text
```

test_integration_flow.py

```python
import base64
from pathlib import Path

from fastapi.testclient import TestClient
from sqlmodel import Session, select

from app.models import QRItem, User


def _auth_headers(client: TestClient) -> dict:
    signup_payload = {
        "email": "flow@example.com",
        "full_name": "Flow User",
        "password": "complexpass123",
    }
    resp = client.post("/api/auth/signup", json=signup_payload)
    assert resp.status_code == 201, resp.text

    login_payload = {
        "email": signup_payload["email"],
        "password": signup_payload["password"],
    }
    resp = client.post("/api/auth/login", json=login_payload)
```

```
    assert resp.status_code == 200, resp.text
    token = resp.json()["access_token"]
    return {"Authorization": f"Bearer {token}"}


def test_full_qr_lifecycle(client: TestClient, engine, tmp_path: Path) -> None:
    headers = _auth_headers(client)
    payload = {
        "title": "Lifecycle",
        "url": "https://example.com",
        "foreground_color": "#111111",
        "background_color": "#ffffff",
        "size": 256,
        "padding": 12,
        "border_radius": 16,
    }

    preview = client.post("/api/qr/preview", json=payload, headers=headers)
    assert preview.status_code == 200, preview.text
    base64.b64decode(preview.json()["png_data"])

    create_resp = client.post("/api/qr", json=payload, headers=headers)
    assert create_resp.status_code == 201, create_resp.text
    created = create_resp.json()

    with Session(engine) as session:
        record = session.exec(select(QRItem)).first()
        assert record is not None
        assert Path(record.svg_path).exists()
        assert Path(record.png_path).exists()

    svg_download = client.get(f"/api/qr/{created['id']}/download", headers=headers)
    assert svg_download.status_code == 200
    assert svg_download.headers["content-type"] == "image/svg+xml"

    png_download = client.get(
        f"/api/qr/{created['id']}/download",
        params={"format": "png"},
        headers=headers,
    )
    assert png_download.status_code == 200
    assert png_download.headers["content-type"] == "image/png"

    delete_resp = client.delete(f"/api/qr/{created['id']}", headers=headers)
    assert delete_resp.status_code == 200
    assert delete_resp.json()["ok"] is True

    with Session(engine) as session:
        assert session.exec(select(QRItem)).all() == []
        assert session.exec(select(User)).first() is not None
```

test_qr.py

```python
from fastapi.testclient import TestClient


def auth_headers(client: TestClient, email: str = "qrtester@example.com") -> dict:
    payload = {
        "email": email,
        "full_name": "QR Tester",
        "password": "strongpass123",
    }
    resp = client.post("/api/auth/signup", json=payload)
    assert resp.status_code == 201
    resp = client.post(
        "/api/auth/login",
        json={"email": payload["email"], "password": payload["password"]},
    )
    assert resp.status_code == 200
    return {"Authorization": f"Bearer {resp.json()['access_token']}"}


def test_create_qr_invalid_payload(client: TestClient) -> None:
    headers = auth_headers(client)
    resp = client.post(
        "/api/qr",
        json={
            "title": "Bad QR",
            "url": "not-a-url",
            "foreground_color": "#000000",
            "background_color": "#ffffff",
            "size": 320,
            "padding": -1,
            "border_radius": 10,
        },
        headers=headers,
    )
    assert resp.status_code == 422


def test_download_requires_authentication(client: TestClient) -> None:
    headers = auth_headers(client)
    payload = {
        "title": "Private",
        "url": "https://example.com",
        "foreground_color": "#000000",
        "background_color": "#ffffff",
        "size": 256,
        "padding": 8,
        "border_radius": 8,
    }
    create_resp = client.post("/api/qr", json=payload, headers=headers)
    assert create_resp.status_code == 201
    qr_id = create_resp.json()["id"]

    resp = client.get(f"/api/qr/{qr_id}/download")
    assert resp.status_code == 401
```

```python
def test_history_only_returns_owner_items(client: TestClient) -> None:
    alice_headers = auth_headers(client)
    client.post(
        "/api/qr",
        json={
            "title": "Alice QR",
            "url": "https://alice.example.com",
            "foreground_color": "#123123",
            "background_color": "#ffffff",
            "size": 256,
            "padding": 8,
            "border_radius": 8,
        },
        headers=alice_headers,
    )

    # create second user
    headers_bob = auth_headers(client, email="bob@example.com")
    resp = client.get("/api/qr/history", headers=headers_bob)
    assert resp.status_code == 200
    assert resp.json() == []

    resp_alice = client.get("/api/qr/history", headers=alice_headers)
    assert len(resp_alice.json()) == 1
```

test_qr_service_unit.py

```python
import base64
from pathlib import Path

from app.services.qr import QRConfig, encode_render, generate_qr_assets, render_qr


def _config() -> QRConfig:
    return QRConfig(
        url="https://example.com",
        foreground_color="#000000",
        background_color="#ffffff",
        size=128,
        padding=4,
        border_radius=0,
    )


def test_render_qr_outputs_svg_and_png() -> None:
    render = render_qr(_config())
    assert "<svg" in render.svg_text
    assert len(render.png_bytes) > 0


def test_encode_render_returns_base64() -> None:
```

```
    render = render_qr(_config())
    preview = encode_render(render)
    assert preview.svg_data.startswith("<svg")
    base64.b64decode(preview.png_data)


def test_generate_qr_assets_writes_files(tmp_path: Path) -> None:
    assets = generate_qr_assets(
        _config(),
        svg_dir=tmp_path / "svgs",
        png_dir=tmp_path / "pngs",
    )
    assert assets.svg_path.exists()
    assert assets.png_path.exists()
    assert assets.svg_path.suffix == ".svg"
    assert assets.png_path.suffix == ".png"
```

test_schemas.py

```
import pytest
from pydantic import ValidationError

from app.schemas import QRCreate


def _base_payload() -> dict:
    return {
        "title": "Valid QR",
        "url": "https://example.com",
        "foreground_color": "#123456",
        "background_color": "#ffffff",
        "size": 256,
        "padding": 8,
        "border_radius": 8,
    }


def test_invalid_hex_colors_raise_validation_error() -> None:
    payload = _base_payload()
    payload["foreground_color"] = "gggggg"
    with pytest.raises(ValidationError):
        QRCreate(**payload)

    payload = _base_payload()
    payload["background_color"] = "#12345"
    with pytest.raises(ValidationError):
        QRCreate(**payload)


def test_invalid_url_rejected() -> None:
    payload = _base_payload()
```

```
    payload["url"] = "notaurl"
    with pytest.raises(ValidationError):
        QRCreate(**payload)


def test_overlay_text_length_enforced() -> None:
    payload = _base_payload()
    payload["overlay_text"] = "TOO-LONG"
    with pytest.raises(ValidationError):
        QRCreate(**payload)


def test_size_constraints_enforced() -> None:
    payload = _base_payload()
    payload["size"] = 32
    with pytest.raises(ValidationError):
        QRCreate(**payload)
```

test_security.py

```
import asyncio
from datetime import timedelta

import pytest
from fastapi import HTTPException
from fastapi.security import HTTPAuthorizationCredentials
from sqlmodel import Session, SQLModel

from app.core.security import (
    create_access_token,
    get_current_user,
    get_password_hash,
    verify_password,
)
from app.db import get_engine
from app.models import User


@pytest.fixture()
def memory_session():
    engine = get_engine("sqlite://")
    SQLModel.metadata.create_all(engine)
    with Session(engine) as session:
        yield session
    SQLModel.metadata.drop_all(engine)


def test_password_hash_roundtrip() -> None:
    raw = "supersafepass"
    hashed = get_password_hash(raw)
    assert hashed != raw
```

```
    assert verify_password(raw, hashed)
    assert not verify_password("wrong", hashed)


def test_create_access_token_contains_subject(memory_session: Session) -> None:
    user = User(
        email="jwt@example.com",
        full_name="",
        hashed_password=get_password_hash("password123"),
    )
    memory_session.add(user)
    memory_session.commit()
    memory_session.refresh(user)

    token = create_access_token(subject=user.id,
expires_delta=timedelta(minutes=5))
    credentials = HTTPAuthorizationCredentials(scheme="Bearer", credentials=token)
    resolved_user = asyncio.run(get_current_user(credentials, memory_session))
    assert resolved_user.id == user.id


def test_get_current_user_invalid_token(memory_session: Session) -> None:
    credentials = HTTPAuthorizationCredentials(scheme="Bearer", credentials="not-a-
jwt")
    with pytest.raises(HTTPException):
        asyncio.run(get_current_user(credentials, memory_session))
```

test_services_exceptions.py

```
from pathlib import Path

import pytest
from sqlmodel import SQLModel

from app.db import get_engine
from app.models import User
from app.schemas import QRCreate
import app.services.qr_items as qr_items


def _make_engine_and_create_tables():
    engine = get_engine("sqlite:///:memory:")
    SQLModel.metadata.create_all(engine)
    return engine


def test_get_owned_item_not_found_raises_http_exception():
    engine = _make_engine_and_create_tables()
    # Create a persisted user and assert get_owned_item raises HTTPException
    from sqlmodel import Session
    from fastapi import HTTPException
```

```
    with Session(engine) as session:
        user = User(email="noone@example.com", full_name="No One",
hashed_password="x")
        session.add(user)
        session.commit()
        session.refresh(user)

        with pytest.raises(HTTPException) as exc:
            qr_items.get_owned_item(session, user, 99999)

        assert exc.value.status_code == 404


def test_create_qr_item_propagates_asset_errors(monkeypatch, tmp_path: Path):
    engine = _make_engine_and_create_tables()
    from sqlmodel import Session

    # create a user
    with Session(engine) as session:
        user = User(email="test@example.com", full_name="T", hashed_password="x")
        session.add(user)
        session.commit()
        session.refresh(user)

    payload = QRCreate(
        title="T",
        url="https://example.com",
        foreground_color="#000000",
        background_color="#ffffff",
        size=128,
        padding=4,
        border_radius=0,
    )

    # Monkeypatch generate_qr_assets to raise an OSError (simulate disk full / IO)
    def _boom(*args, **kwargs):
        raise OSError("disk full")

    monkeypatch.setattr(qr_items, "generate_qr_assets", _boom)

    with Session(engine) as session:
        with pytest.raises(OSError):
            qr_items.create_qr_item(
                session,
                payload,
                user,
                svg_dir=tmp_path / "svgs",
                png_dir=tmp_path / "pngs",
            )
```